

JENKINS-TASK-01

1.Install Jenkins and run Jenkins on port number 8081.

Jenkins Installation and Configuration on Port 8081

Objective: To install Jenkins on an Amazon Linux 2023 EC2 instance, configure Jenkins to run on **port 8081**, start the Jenkins service, and access the Jenkins dashboard through a web browser.

Prerequisites

Before installing Jenkins, make sure you have:

- AWS account
- EC2 instance
- Amazon Linux 2023
- SSH access to the EC2 instance
- Internet connectivity from the EC2 instance
- Security Group access
- Java 21
- Port **8081** allowed in the Security Group

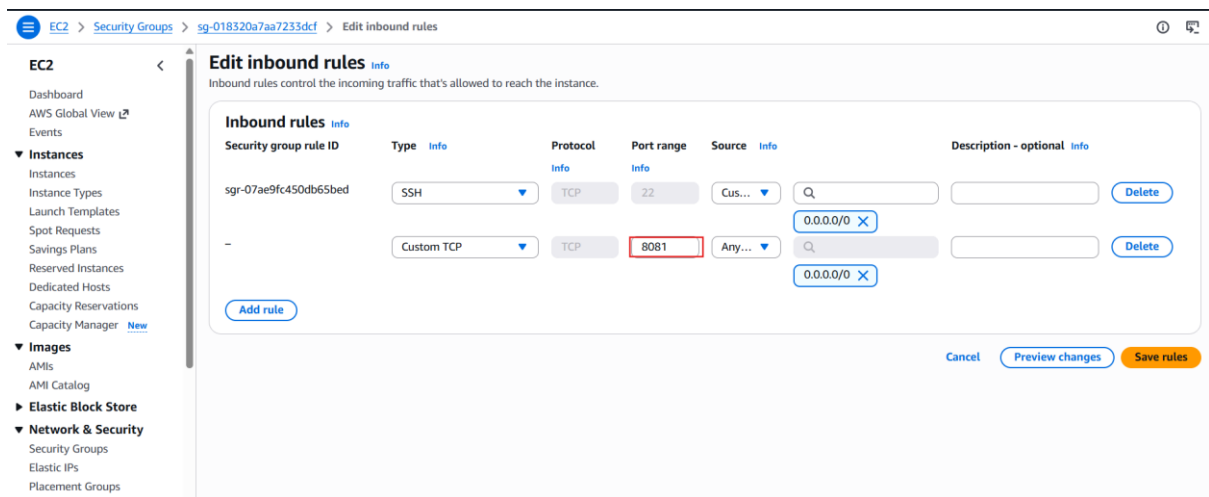
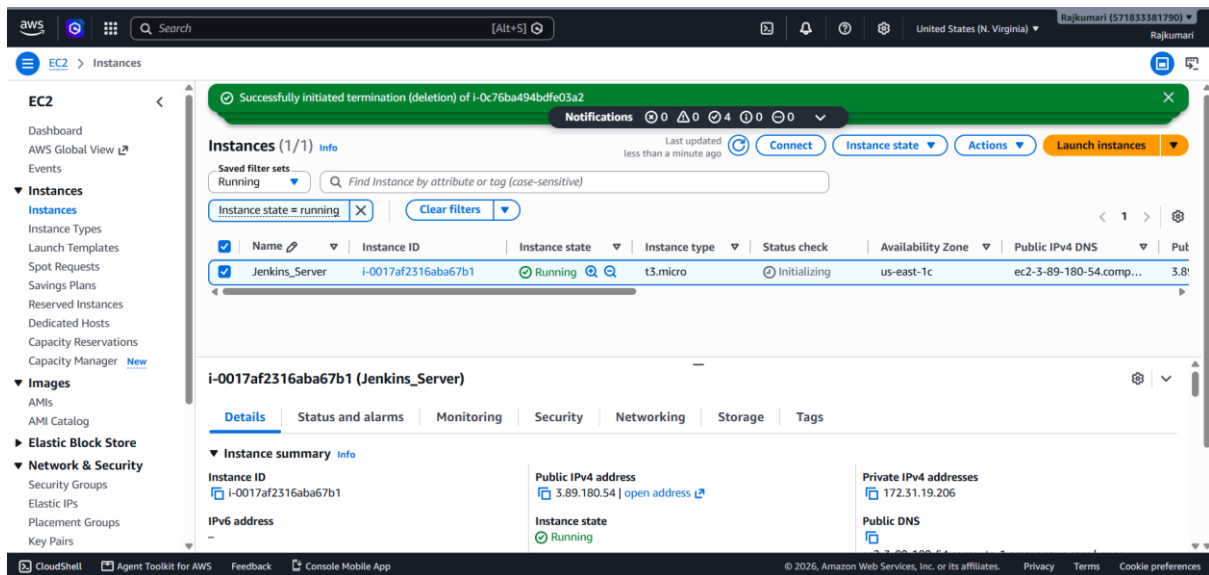
Step 1 – Connect to the EC2 Instance

Open your AWS Console.

Go to: **EC2 → Instances → Select your EC2 Instance → Connect**

Select: **EC2 Instance Connect**

or connect using SSH.



Step 2 – Update the System Packages

Run: `sudo dnf update -y`

Step 3 – Install Java 21

Jenkins requires Java to run. Current Jenkins LTS releases support Java 21, making it a suitable choice for this assignment.

Run: `sudo dnf install java-21-amazon-corretto -y`

Step 4 – Verify Java Installation

Run: `java -version`

```
root@ip-172-31-19-206:~
[root@ip-172-31-19-206 ~]# sudo dnf update -y
Amazon Linux 2023 Kernel Livepatch repository 542 kB/s | 69 kB 00:00
Last metadata expiration check: 0:00:01 ago on Tue Aug 11 07:04:53 2026.
Dependencies resolved.
Nothing to do.
Complete!
[root@ip-172-31-19-206 ~]# sudo dnf install java-21-amazon-corretto -y
Last metadata expiration check: 0:00:18 ago on Tue Aug 11 07:04:53 2026.
Dependencies resolved.
=====
Package Arch Version Repository Size
=====
Installing:
java-21-amazon-corretto x86_64 1:21.0.12+8-1.amzn2023.1 amazonlinux 218 k
Installing dependencies:
alsa-lib x86_64 1.2.7.2-1.amzn2023.0.3 amazonlinux 503 k
cairo x86_64 1.18.0-4.amzn2023.0.3 amazonlinux 717 k
dejavu-sans-fonts noarch 2.37-16.amzn2023.0.2 amazonlinux 1.3 M
dejavu-sans-mono-fonts noarch 2.37-16.amzn2023.0.2 amazonlinux 467 k
dejavu-serif-fonts noarch 2.37-16.amzn2023.0.2 amazonlinux 1.0 M
fontconfig x86_64 2.13.94-2.amzn2023.0.2 amazonlinux 273 k
fontfiles x86_64 1:2.0.5-12.amzn2023.0.2.noarch amazonlinux 8.5 k
fonttools x86_64 4.52.0-1.amzn2023.0.2.noarch amazonlinux 1.1 M
freetype x86_64 2.13.2-5.amzn2023.0.2.x86_64 amazonlinux 1.1 M
glibc x86_64 2.37-16.amzn2023.0.2.x86_64 amazonlinux 1.1 M
google-noto-fonts-common-20240401-1.amzn2023.0.2.noarch amazonlinux 1.1 M
google-noto-sans-vf-fonts-20240401-1.amzn2023.0.2.noarch amazonlinux 1.1 M
graphite2-1.3.14-7.amzn2023.0.3.x86_64 amazonlinux 1.1 M
harfbuzz-7.0.0-2.amzn2023.0.2.x86_64 amazonlinux 1.1 M
java-21-amazon-corretto-1:21.0.12+8-1.amzn2023.1.x86_64 amazonlinux 218 k
java-21-amazon-corretto-headless-1:21.0.12+8-1.amzn2023.1.x86_64 amazonlinux 218 k
javapackages-filesystem-6.0.0-7.amzn2023.0.6.noarch amazonlinux 1.1 M
langpacks-core-font-en-3.0-21.amzn2023.0.4.noarch amazonlinux 1.1 M
libICE-1.1.1-3.amzn2023.0.1.x86_64 amazonlinux 1.1 M
libSM-1.2.4-3.amzn2023.0.1.x86_64 amazonlinux 1.1 M
libX11-1.8.10-2.amzn2023.0.1.x86_64 amazonlinux 1.1 M
libX11-common-1.8.10-2.amzn2023.0.1.noarch amazonlinux 1.1 M
libXau-1.0.11-6.amzn2023.0.1.x86_64 amazonlinux 1.1 M
libXext-1.3.6-1.amzn2023.0.1.x86_64 amazonlinux 1.1 M
libXi-1.8.2-1.amzn2023.0.1.x86_64 amazonlinux 1.1 M
libXinerama-1.1.5-6.amzn2023.0.1.x86_64 amazonlinux 1.1 M
libXrandr-1.5.4-3.amzn2023.0.1.x86_64 amazonlinux 1.1 M
libXrender-0.9.11-6.amzn2023.0.1.x86_64 amazonlinux 1.1 M
libXt-1.3.0-3.amzn2023.0.1.x86_64 amazonlinux 1.1 M
libXtst-1.2.5-1.amzn2023.0.1.x86_64 amazonlinux 1.1 M
libbrotli-1.0.9-4.amzn2023.0.2.x86_64 amazonlinux 1.1 M
libjpeg-turbo-2.1.4-2.amzn2023.0.5.x86_64 amazonlinux 1.1 M
libpng-2:1.6.37-10.amzn2023.0.13.x86_64 amazonlinux 1.1 M
libxcb-1.17.0-1.amzn2023.0.1.x86_64 amazonlinux 1.1 M
pixman-0.43.4-1.amzn2023.0.4.x86_64 amazonlinux 1.1 M
xml-common-0.6.3-56.amzn2023.0.2.noarch amazonlinux 1.1 M
=====
Verifying : libXext-1.3.6-1.amzn2023.0.1.x86_64 23/35
Verifying : libXi-1.8.2-1.amzn2023.0.1.x86_64 24/35
Verifying : libXinerama-1.1.5-6.amzn2023.0.1.x86_64 25/35
Verifying : libXrandr-1.5.4-3.amzn2023.0.1.x86_64 26/35
Verifying : libXrender-0.9.11-6.amzn2023.0.1.x86_64 27/35
Verifying : libXt-1.3.0-3.amzn2023.0.1.x86_64 28/35
Verifying : libXtst-1.2.5-1.amzn2023.0.1.x86_64 29/35
Verifying : libbrotli-1.0.9-4.amzn2023.0.2.x86_64 30/35
Verifying : libjpeg-turbo-2.1.4-2.amzn2023.0.5.x86_64 31/35
Verifying : libpng-2:1.6.37-10.amzn2023.0.13.x86_64 32/35
Verifying : libxcb-1.17.0-1.amzn2023.0.1.x86_64 33/35
Verifying : pixman-0.43.4-1.amzn2023.0.4.x86_64 34/35
Verifying : xml-common-0.6.3-56.amzn2023.0.2.noarch 35/35
Installed:
alsa-lib-1.2.7.2-1.amzn2023.0.3.x86_64
cairo-1.18.0-4.amzn2023.0.3.x86_64
dejavu-sans-fonts-2.37-16.amzn2023.0.2.noarch
dejavu-sans-mono-fonts-2.37-16.amzn2023.0.2.noarch
dejavu-serif-fonts-2.37-16.amzn2023.0.2.noarch
fontconfig-2.13.94-2.amzn2023.0.2.x86_64
fontfiles-1:2.0.5-12.amzn2023.0.2.noarch
freetype-2.13.2-5.amzn2023.0.2.x86_64
glibc-2.37-16.amzn2023.0.2.x86_64
google-noto-fonts-common-20240401-1.amzn2023.0.2.noarch
google-noto-sans-vf-fonts-20240401-1.amzn2023.0.2.noarch
graphite2-1.3.14-7.amzn2023.0.3.x86_64
harfbuzz-7.0.0-2.amzn2023.0.2.x86_64
java-21-amazon-corretto-1:21.0.12+8-1.amzn2023.1.x86_64
java-21-amazon-corretto-headless-1:21.0.12+8-1.amzn2023.1.x86_64
javapackages-filesystem-6.0.0-7.amzn2023.0.6.noarch
langpacks-core-font-en-3.0-21.amzn2023.0.4.noarch
libICE-1.1.1-3.amzn2023.0.1.x86_64
libSM-1.2.4-3.amzn2023.0.1.x86_64
libX11-1.8.10-2.amzn2023.0.1.x86_64
libX11-common-1.8.10-2.amzn2023.0.1.noarch
libXau-1.0.11-6.amzn2023.0.1.x86_64
libXext-1.3.6-1.amzn2023.0.1.x86_64
libXi-1.8.2-1.amzn2023.0.1.x86_64
libXinerama-1.1.5-6.amzn2023.0.1.x86_64
libXrandr-1.5.4-3.amzn2023.0.1.x86_64
libXrender-0.9.11-6.amzn2023.0.1.x86_64
libXt-1.3.0-3.amzn2023.0.1.x86_64
libXtst-1.2.5-1.amzn2023.0.1.x86_64
libbrotli-1.0.9-4.amzn2023.0.2.x86_64
libjpeg-turbo-2.1.4-2.amzn2023.0.5.x86_64
libpng-2:1.6.37-10.amzn2023.0.13.x86_64
libxcb-1.17.0-1.amzn2023.0.1.x86_64
pixman-0.43.4-1.amzn2023.0.4.x86_64
xml-common-0.6.3-56.amzn2023.0.2.noarch
Complete!
[root@ip-172-31-19-206 ~]# java -version
openjdk version "21.0.12" 2026-07-21 LTS
OpenJDK Runtime Environment Corretto-21.0.12.8.1 (build 21.0.12+8-LTS)
OpenJDK 64-Bit Server VM Corretto-21.0.12.8.1 (build 21.0.12+8-LTS, mixed mode,
sharing)
[root@ip-172-31-19-206 ~]#
```

Step 5 – Add the Jenkins Repository

Download the Jenkins LTS repository:

```
sudo wget -O /etc/yum.repos.d/jenkins.repo \
https://pkg.jenkins.io/rpm-stable/jenkins.repo
```

The official Jenkins documentation provides the RPM repository method for Red Hat-based Linux distributions.

Step 6 – Import Jenkins Repository Key

Run: `sudo rpm --import https://pkg.jenkins.io/rpm-stable/jenkins.io-2026.key`

This imports the Jenkins repository signing key so that packages can be verified.

The current Jenkins AWS installation tutorial provides this repository-key step.

Step 7 – Refresh Package Information

Run: `sudo dnf upgrade -y`

This refreshes and upgrades packages using the configured repositories.

```
root@ip-172-31-19-206:~
[root@ip-172-31-19-206 ~]# sudo wget -O /etc/yum.repos.d/jenkins.repo \
https://pkg.jenkins.io/rpm-stable/jenkins.repo
--2026-08-11 07:09:03-- https://pkg.jenkins.io/rpm-stable/jenkins.repo
Resolving pkg.jenkins.io (pkg.jenkins.io)... 146.75.38.133, 2a04:4e42:77::645
Connecting to pkg.jenkins.io (pkg.jenkins.io)|146.75.38.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 267 [application/octet-stream]
Saving to: '/etc/yum.repos.d/jenkins.repo'

/etc/yum.repos.d/jenkins.repo 100%[=====] 267 --.-KB/s in 0s

2026-08-11 07:09:03 (22.5 MB/s) - '/etc/yum.repos.d/jenkins.repo' saved [267/267]

[root@ip-172-31-19-206 ~]# sudo rpm --import https://pkg.jenkins.io/rpm-stable/jenkins.io-2026.key
[root@ip-172-31-19-206 ~]# sudo dnf upgrade -y
Jenkins-stable 12 kB/s | 833 B 00:00
Jenkins-stable 109 kB/s | 1.6 kB 00:00
Importing GPG key 0x14ABFC68:
  Userid : "Jenkins Project <jenkinsci-board@googlegroups.com>"
  Fingerprint: 5E38 6EAD B55F 0150 4CAE 8BCF 7198 F4B7 14AB FC68
  From : https://pkg.jenkins.io/rpm-stable/repodata/repomd.xml.key
Jenkins-stable 38 kB/s | 2.6 kB 00:00
Dependencies resolved.
Nothing to do.
Complete!
[root@ip-172-31-19-206 ~]# |
```

Step 8 – Install Jenkins

Run: `sudo dnf install jenkins -y`

Wait until the installation finishes.

Jenkins will be installed as a system service.

The Jenkins package creates a dedicated jenkins user and configures Jenkins as a service managed by system

Step 9 – Verify Jenkins Installation

Check the Jenkins service: `sudo systemctl status Jenkins`

```
root@ip-172-31-19-206:~  
[root@ip-172-31-19-206 ~]# sudo dnf install jenkins -y  
Last metadata expiration check: 0:02:01 ago on Tue Aug 11 07:09:32 2026.  
Dependencies resolved.  
=====
```

Package	Architecture	Version	Repository	Size
---------	--------------	---------	------------	------

```
=====
```

Installing:

jenkins	noarch	2.568.2-1	jenkins	96 M
---------	--------	-----------	---------	------

Transaction Summary

```
=====
```

Install 1 Package

Total download size: 96 M
Installed size: 96 M
Downloading Packages:

jenkins-2.568.2-1.noarch.rpm	108 MB/s 96 MB	00:00
------------------------------	------------------	-------

```
-----
```

Total 108 MB/s | 96 MB 00:00

Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction

Preparing	:	1/1
Running scriptlet:	jenkins-2.568.2-1.noarch	1/1
Installing	:	1/1
Running scriptlet:	jenkins-2.568.2-1.noarch	1/1
Verifying	:	1/1

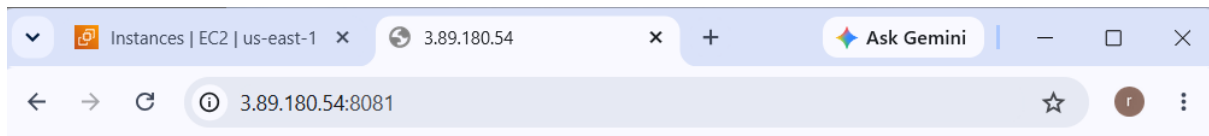
Installed:

jenkins-2.568.2-1.noarch

Complete!

```
[root@ip-172-31-19-206 ~]# sudo systemctl status jenkins  
o jenkins.service - Jenkins Continuous Integration Server  
   Loaded: loaded (/usr/lib/systemd/system/jenkins.service; disabled; preset:  
   Active: inactive (dead)
```

```
[root@ip-172-31-19-206 ~]# sudo systemctl start jenkins  
[root@ip-172-31-19-206 ~]# sudo systemctl status jenkins  
● jenkins.service - Jenkins Continuous Integration Server  
   Loaded: loaded (/usr/lib/systemd/system/jenkins.service; disabled; preset:  
   Active: active (running) since Tue 2026-08-11 07:13:05 UTC; 5s ago  
 Main PID: 26683 (java)  
    Tasks: 42 (limit: 1059)  
  Memory: 393.1M  
     CPU: 20.174s  
   CGroup: /system.slice/jenkins.service  
           └─26683 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/jav
```



This site can't be reached

3.89.180.54 took too long to respond.

Step 10 – Configure Jenkins to Run on Port 8081

This is the **important step** for your assignment.

By default Jenkins uses: 8080

You need to change it to: 8081

Run: `sudo systemctl edit Jenkins`

```
root@ip-172-31-19-206:~
GNU nano 8.3 /etc/systemd/system/jenkins.service.d/override.conf075e1792ba39823e
# StartLimitBurst=5
# StartLimitIntervalSec=5m
#
# [Service]
# Type=notify
# NotifyAccess=main
# ExecStart=/usr/bin/jenkins
# Restart=on-failure
# SuccessExitStatus=143
# Environment="JENKINS_PORT=8081"
#
# # Configures the time to wait for start-up. If Jenkins does not signal start-up
# # completion within the configured time, the service will be considered failed
# # and will be shut down again. Takes a unit-less value in seconds, or a time span
# # value such as "5min 20s". Pass "infinity" to disable the timeout logic.
# TimeoutStartSec=90
#
# # Unix account that runs the Jenkins daemon
# # Be careful when you change this, as you need to update the permissions of
# # $JENKINS_HOME, $JENKINS_LOG, and (if you have already run Jenkins)
# # $JENKINS_WEBROOT.
# User=jenkins
# Group=jenkins
#
# # Directory where Jenkins stores its configuration and workspaces
# Environment="JENKINS_HOME=/var/lib/jenkins"
# WorkingDirectory=/var/lib/jenkins
#
# # Location of the Jenkins WAR
# #Environment="JENKINS_WAR=/usr/share/java/jenkins.war"
#
# # Location of the exploded WAR
# Environment="JENKINS_WEBROOT=/usr/share/java/jenkins.war"
#
# # Location of the Jenkins log. By default, systemd-journald(8) is used.
# #Environment="JENKINS_LOG=/var/log/jenkins/jenkins.log"
#
# # The Java home directory. When left empty, JENKINS_JAVA_CMD and PATH are consulted.
# #Environment="JAVA_HOME=/usr/lib/jvm/java-21-openjdk-amd64"
#
# # The Java executable. When left empty, JAVA_HOME and PATH are consulted.
# #Environment="JENKINS_JAVA_CMD=/etc/alternatives/java"
#
# # Arguments for the Jenkins JVM
# Environment="JAVA_OPTS=-Djava.awt.headless=true"
#
# # Unix Domain Socket to listen on for local HTTP requests. Default is disabled.
# #Environment="JENKINS_UNIX_DOMAIN_PATH=/run/jenkins/jenkins.socket"
#
# Help
# Exit
# Write Out
# Read File
# Where Is
# Replace
# Cut
# Paste
# Execute
# Justify
# Location
# Go To Line
# Undo
# Redo
# Set Mark
# Copy
# To Bracket
# where was
# Previous
# Next
# Back
# Forward
```

Step 11 – Reload systemd

Run: `sudo systemctl daemon-reload`

Step 12 – Enable Jenkins at System Startup

Run: `sudo systemctl enable Jenkins`

Step 13 – Start Jenkins

Run: `sudo systemctl start Jenkins`

Run: `sudo systemctl status Jenkins`

Step 14 – Verify Jenkins Port

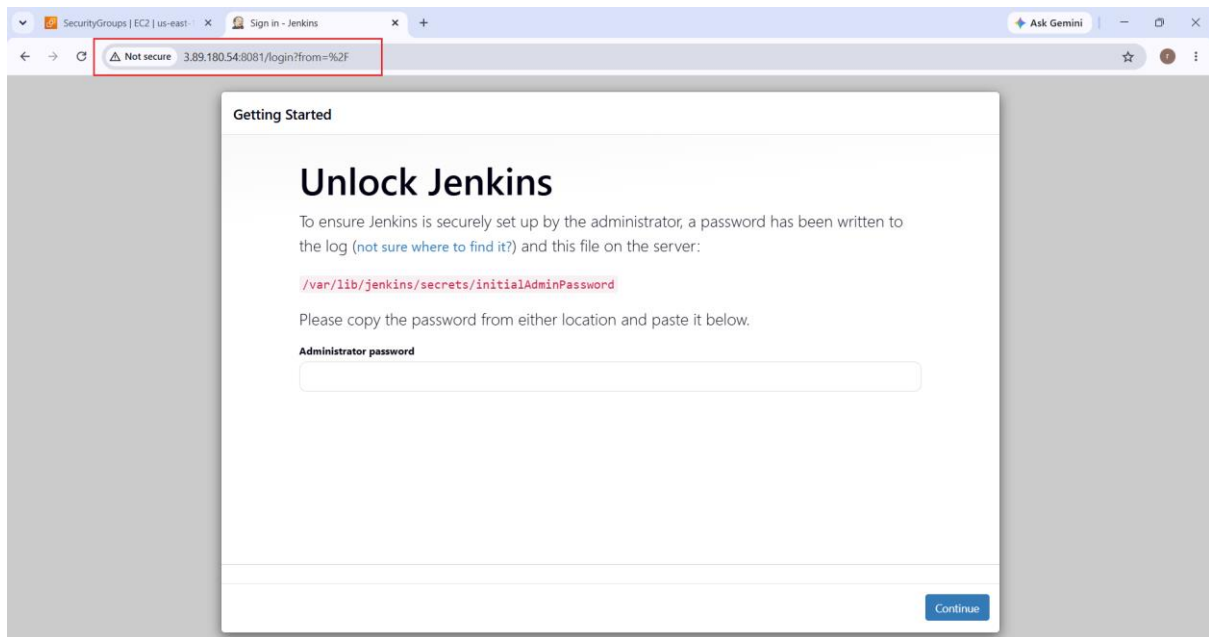
Now verify that Jenkins is actually listening on 8081.

Run: `sudo ss -lntp | grep 8081`

```
root@ip-172-31-19-206:~
[root@ip-172-31-19-206 ~]# sudo mkdir -p /etc/systemd/system/jenkins.service.d
[root@ip-172-31-19-206 ~]# sudo tee /etc/systemd/system/jenkins.service.d/override.conf > /dev/null <<'EOF'
[Service]
Environment="JENKINS_PORT=8081"
EOF
[root@ip-172-31-19-206 ~]# ll
total 0
[root@ip-172-31-19-206 ~]#
[root@ip-172-31-19-206 ~]# sudo cat /etc/systemd/system/jenkins.service.d/override.conf
[Service]
Environment="JENKINS_PORT=8081"
[root@ip-172-31-19-206 ~]# sudo systemctl daemon-reload
[root@ip-172-31-19-206 ~]# sudo systemctl restart jenkins
[root@ip-172-31-19-206 ~]# sudo systemctl status jenkins
● jenkins.service - Jenkins Continuous Integration Server
   Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled;
   Drop-In: /etc/systemd/system/jenkins.service.d
            └─override.conf
   Active: active (running) since Tue 2026-08-11 07:42:50 UTC; 17s ago
   Main PID: 28265 (java)
   Tasks: 41 (limit: 1059)
   Memory: 259.7M
   CPU: 17.018s
   CGroup: /system.slice/jenkins.service
            └─28265 /usr/bin/java -Djava.awt.headless=true -jar /usr/s

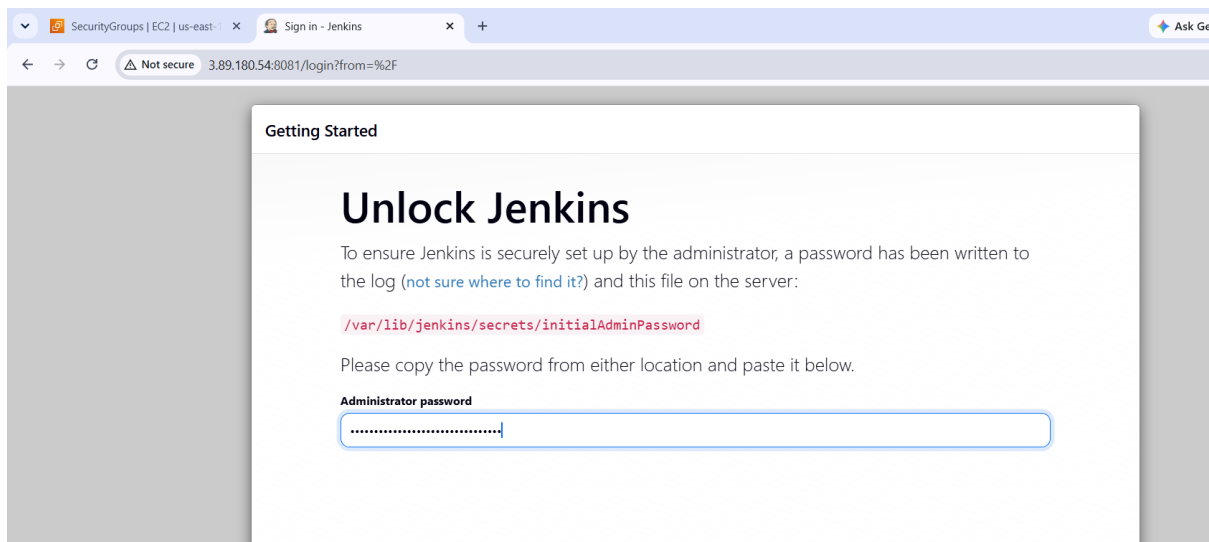
Aug 11 07:42:46 ip-172-31-19-206.ec2.internal jenkins[28265]: [LF]>
Aug 11 07:42:46 ip-172-31-19-206.ec2.internal jenkins[28265]: [LF]> Thi
Aug 11 07:42:46 ip-172-31-19-206.ec2.internal jenkins[28265]: [LF]>
Aug 11 07:42:46 ip-172-31-19-206.ec2.internal jenkins[28265]: [LF]> ***
Aug 11 07:42:46 ip-172-31-19-206.ec2.internal jenkins[28265]: [LF]> ***
Aug 11 07:42:46 ip-172-31-19-206.ec2.internal jenkins[28265]: [LF]> ***
Aug 11 07:42:50 ip-172-31-19-206.ec2.internal jenkins[28265]: 2026-08-1
Aug 11 07:42:50 ip-172-31-19-206.ec2.internal jenkins[28265]: 2026-08-1
Aug 11 07:42:50 ip-172-31-19-206.ec2.internal systemd[1]: Started jenkins
Aug 11 07:42:55 ip-172-31-19-206.ec2.internal jenkins[28265]: 2026-08-1

[root@ip-172-31-19-206 ~]# sudo ss -lntp | grep 8081
LISTEN 0      50          *:8081      *:~        users:((("java"
,pid=28265,fd=9))
[root@ip-172-31-19-206 ~]# |
```



Step 15 – Retrieve Initial Administrator Password

On the EC2 instance, run: `sudo cat /var/lib/jenkins/secrets/initialAdminPassword`



← → ↻ Not secure 3.89.180.54:8081

Getting Started

Create First Admin User

Username
rajkumari

Password

Confirm password

Full name
Badugu Rajkumari

Email
brajkumari7675@gmail.com

Jenkins 2.568.2

[Skip and continue as admin](#) [Save and Continue](#)

← → ↻ Not secure 3.89.180.54:8081

Getting Started

Instance Configuration

Jenkins URL:

The Jenkins URL is used to provide the root URL for absolute links to various Jenkins resources. That means this value is required for proper operation of many Jenkins features including email notifications, PR status updates, and the BUILD_URL environment variable provided to build steps.

The proposed default value shown is **not saved yet** and is generated from the current request, if possible. The best practice is to set this value to the URL that users are expected to use. This will avoid confusion when sharing or viewing links.

← → ↻ Not secure 3.89.180.54:8081/user/rajkumari/

 **Jenkins** / Badugu Rajkumari

 **Badugu Rajkumari** [Add description](#)

Profile

Builds

My Views

Account

Appearance

Preferences

Security

Experiments

Credentials

Username: rajkumari

REST API Jenkins 2.568.2

2. Secure Jenkins server

Objective: To secure the Jenkins server by configuring authentication, authorization, CSRF protection, Security Group restrictions, secure administrator access, and other recommended Jenkins security settings.

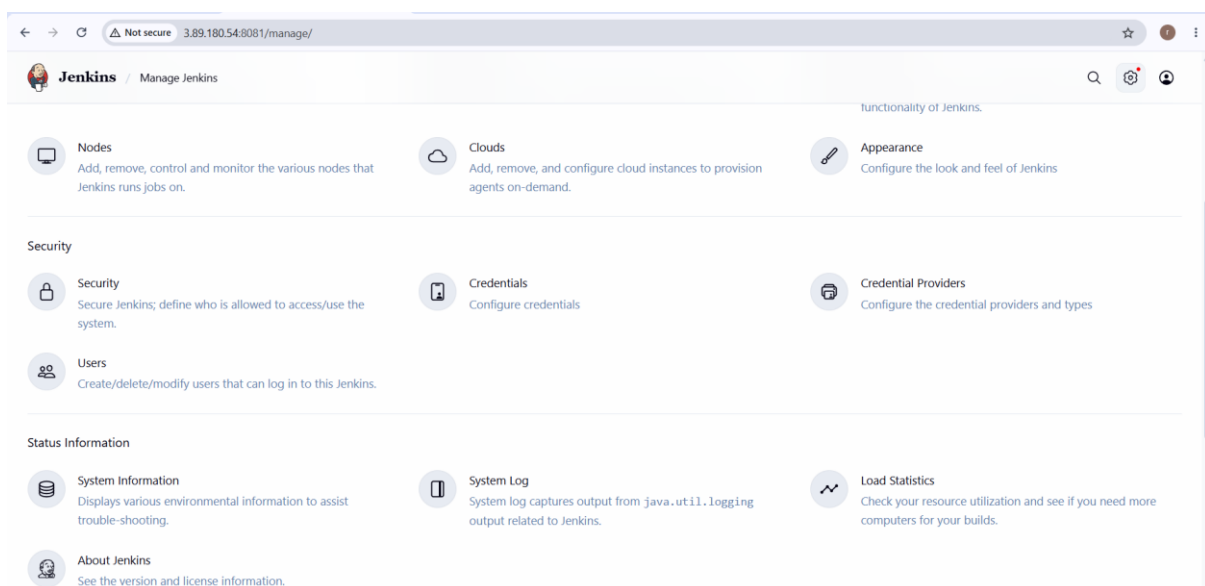
Step 1 – Verify Jenkins Security Is Enabled

Open Jenkins: `http://<EC2-PUBLIC-IP>:8081`

Log in using your Jenkins administrator account.

Go to: **Manage Jenkins** → **Security**

Depending on your Jenkins version, the exact menu may appear as **Manage Jenkins** → **Security** or under the security configuration area.



Step 2 – Configure Authentication

Authentication answers:

Who are you?

Jenkins should not allow anonymous users to administer the server.

In the Security configuration, verify that Jenkins uses:

Security Realm → Jenkins' own user database

For a basic lab environment, this is sufficient.

Make sure:

Allow users to sign up

is **disabled** unless you specifically need users to create their own accounts

 **Jenkins** / Manage Jenkins ▾ / Security 🔍 ⚙️ 👤

Security

Secure Jenkins; define who is allowed to access/use the system.

Authentication

☐ Disable "Keep me signed in" ?

Security Realm

Jenkins' own user database ▾

☐ Allow users to sign up ?

Step 3 – Configure Authorization

Authorization answers:

What are you allowed to do?

Under **Authorization**, use a role-based or matrix-based authorization strategy where appropriate.

For a simple assignment, you can use:

Logged-in users can do anything

This means:

- Anonymous users → No administrative access
- Authenticated users → Jenkins access according to the selected strategy

For a production environment, use **least privilege** rather than allowing every authenticated user to do everything.

Authorization

Logged-in users can do anything

☐

Allow anonymous read access ?

Verify CSRF Protection

CSRF means:

Cross-Site Request Forgery

It is an attack where a malicious website attempts to make an authenticated user perform an unwanted action.

Jenkins provides CSRF protection using a **crumb** mechanism.

In the Jenkins security configuration, verify that:

CSRF Protection

is enabled.

Do **not** disable CSRF protection simply to make an integration work. Instead, configure the integration correctly.

CSRF Protection

Crumb Issuer

Default Crumb Issuer



Git plugin notifyCommit access tokens

Current access tokens ?

There are no access tokens yet.

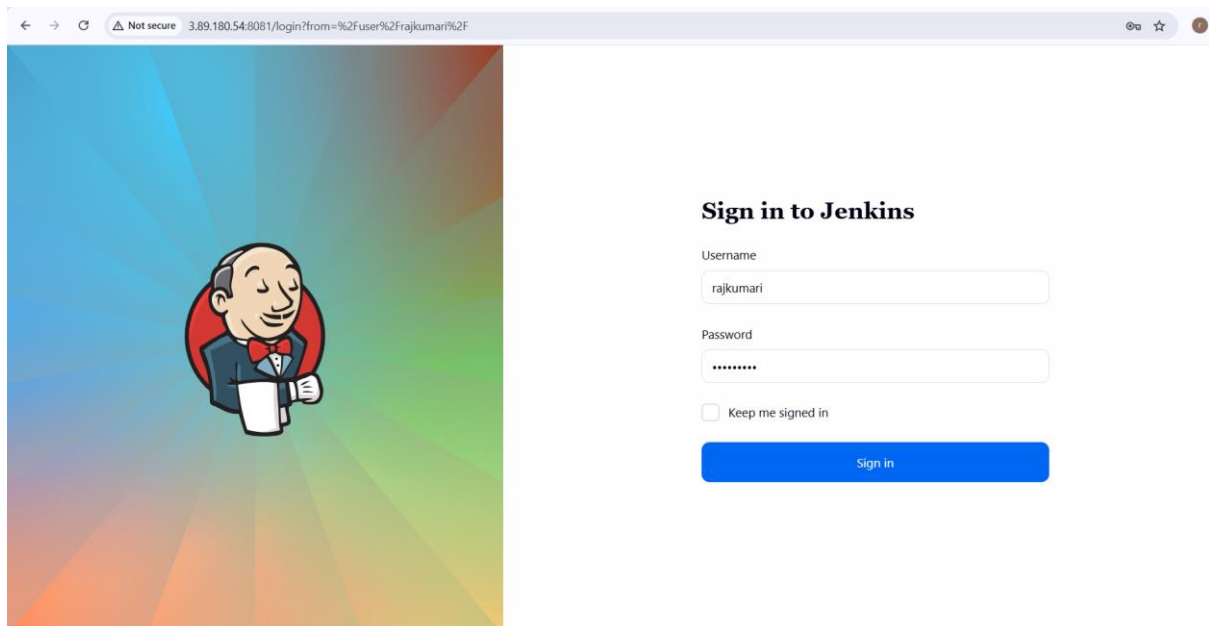
+ Add new access token

Step 5 – Secure the Jenkins Administrator Account

Go to:

Jenkins → Your Username → Configure

Use a strong password.



Step 6 – Secure the AWS Security Group

This is one of the most important security controls for your EC2 Jenkins server.

Go to:

AWS Console → EC2 → Instances → Your Jenkins Instance → Security → Security Groups

Select:

Inbound rules → Edit inbound rules

Your SSH rule should preferably be:

Type	Port	Source
SSH	22	My IP

Custom TCP 8081 My IP

Avoid this for Jenkins:

8081 → 0.0.0.0/0

because it makes Jenkins accessible from the entire Internet.

For a learning environment, **My IP** is much safer.

Step 10 – Keep Jenkins Updated

Regularly check: **Manage Jenkins → Plugins**

and: **Manage Jenkins → System**

Keep Jenkins and installed plugins updated.

This is important because Jenkins plugins can introduce security vulnerabilities if they are outdated.

Only install plugins that you actually need.

Step 11 – Install Only Required Plugins

Go to: **Manage Jenkins → Plugins**

Review installed plugins.

Remove unnecessary plugins when appropriate.

A good security principle is: **Install the minimum number of plugins required for your CI/CD pipeline.**

Every additional plugin increases the software surface that needs to be maintained and secured.

3. Create users called DevOps, Testing in Jenkins with Limited access.

Step 1: Open Jenkins in your web browser and log in using an account with **Administrator** privileges.

Step 2: Open Manage Jenkins

From the Jenkins dashboard, click:

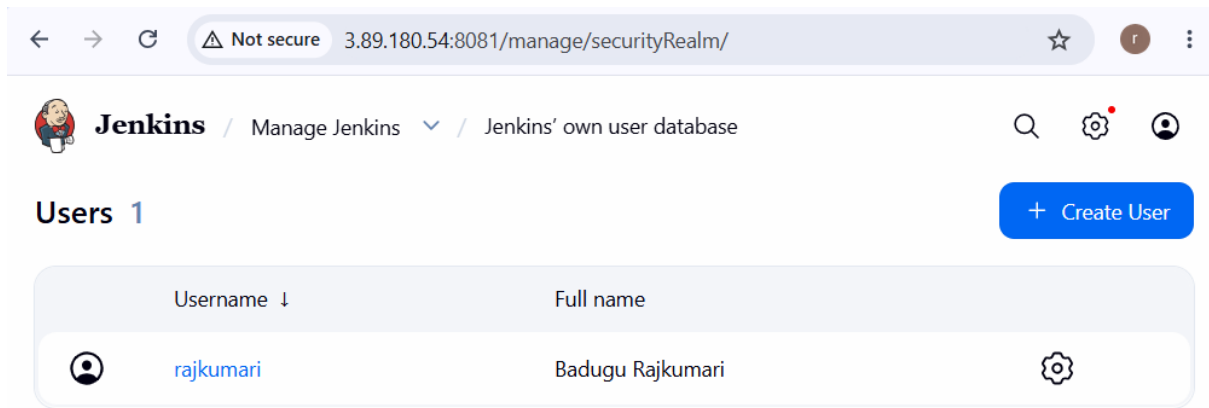
Manage Jenkins

Step 3: Open Users

Under the **Security** or **System Configuration** section, select:

Users

If **Users** is not directly visible, go to **Manage Jenkins → Users**.



Step 4: Create the DevOps User

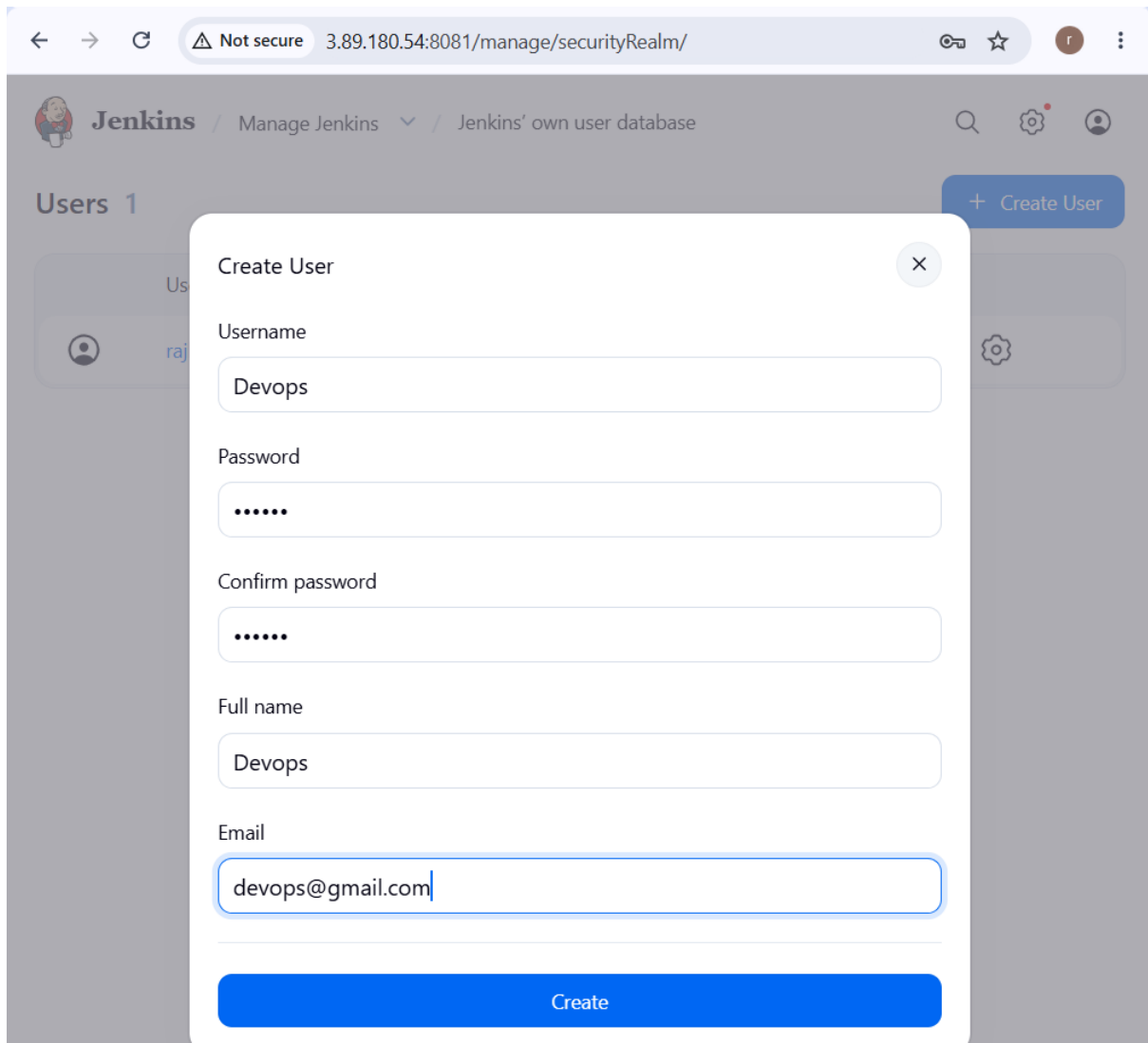
Click:

Create User

Enter the following details:

- **Username:** DevOps
- **Password:** Enter a strong password
- **Confirm password:** Re-enter the password
- **Full name:** DevOps
- **Email address:** Optional

Click **Create User**.



Step 5: Create the Testing User

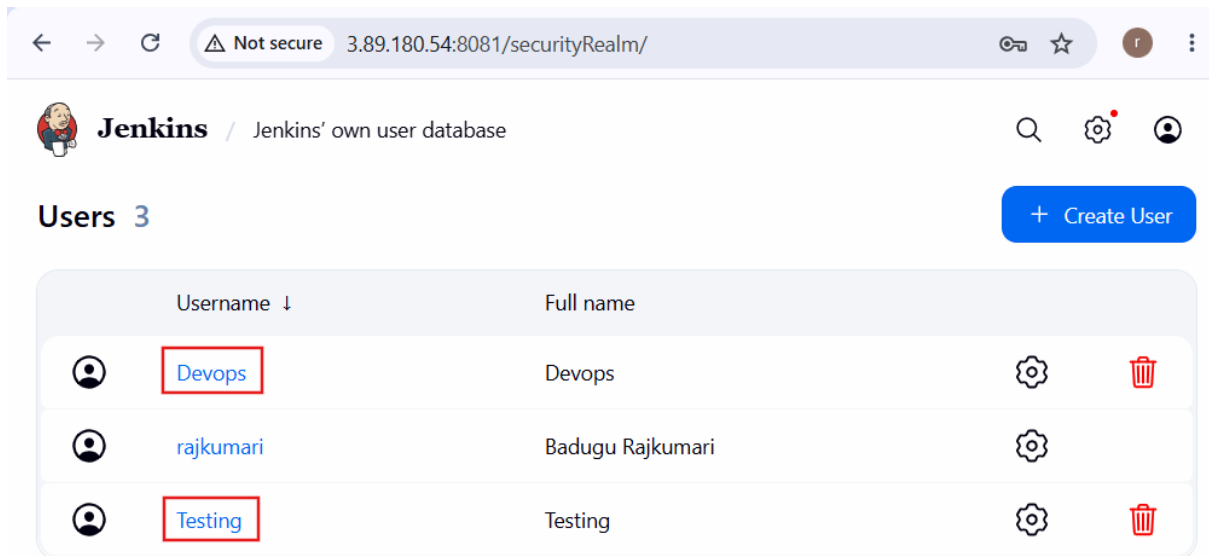
Again, click:

Create User

Enter:

- **Username:** Testing
- **Password:** Enter a strong password
- **Confirm password:** Re-enter the password
- **Full name:** Testing
- **Email address:** Optional

Click **Create User**.

A screenshot of the Jenkins web interface showing the 'Users' page. The browser address bar shows '3.89.180.54:8081/securityRealm/'. The page title is 'Jenkins / Jenkins' own user database'. There are three users listed in a table: 'Devops', 'rajkumari', and 'Testing'. The 'Devops' and 'Testing' usernames are highlighted with red boxes. A '+ Create User' button is in the top right.

	Username ↓	Full name		
	Devops	Devops		
	rajkumari	Badugu Rajkumari		
	Testing	Testing		

Step 6: Open Security Configuration

Return to:

Manage Jenkins → Security

Locate the **Authorization** section.

Step 7: Select Matrix-Based Security

Under **Authorization**, select:

Matrix-based security

This allows you to provide different permissions to different Jenkins users.

If **Matrix-based security** is not available, install the **Matrix Authorization Strategy** plugin through **Manage Jenkins → Plugins**.

← → ↻ ⚠ Not secure 3.89.180.54:8081/manage/configureSecurity/ ☆ Ⓜ ⋮

Jenkins / Manage Jenkins ▾ / Security 🔍 ⚙️ 👤

Authentication

☐ Disable "Keep me signed in" ?

Security Realm

Jenkins' own user database ▾

☐ Allow users to sign up ?

Authorization

Matrix-based security ▾

🔍 Filter users and groups ≡ Add user Add group

👤 Anonymous	No permissions granted	☑️ 📄 ▾
👥 Authenticated Users	No permissions granted	☑️ 📄 ▾

Markup Formatter

Markup Formatter ?

Plain text ▾

Step 8: Add the DevOps User

In the authorization matrix, click **Add user or group**.

Enter:

DevOps

Select the DevOps user from the list.

Step 9: Give Limited DevOps Permissions

Grant only the permissions required for DevOps activities.

For example:

- Overall → Read ☑️

- Job → Read ✓
- Job → Build ✓
- Job → Workspace ✓
- Job → Cancel ✓

The screenshot shows the Jenkins 'Manage Jenkins' > 'Security' page. The 'Matrix-based security' section is active. A search bar 'Filter users and groups' is at the top. Below it, the 'Devops' user is selected, with a summary 'Overall: Read · Job: Build, Cancel, Read, Workspace'. The permissions are listed in a table:

Category	Permission	Value
Overall	Administer	Read
	Manage	Read
Credentials	Create	
	Delete	
	ManageDomains	
	Update	
Agent	Build	
	Configure	
	Connect	
	Create	
Job	Build	
	Cancel	
	Configure	
	Create	
Run	Delete	
	Discover (implied)	

Additional permissions shown in blue buttons: Read, Build, Cancel, Workspace.

Step 10: Add the Testing User

Click **Add user or group** again and enter:

Testing

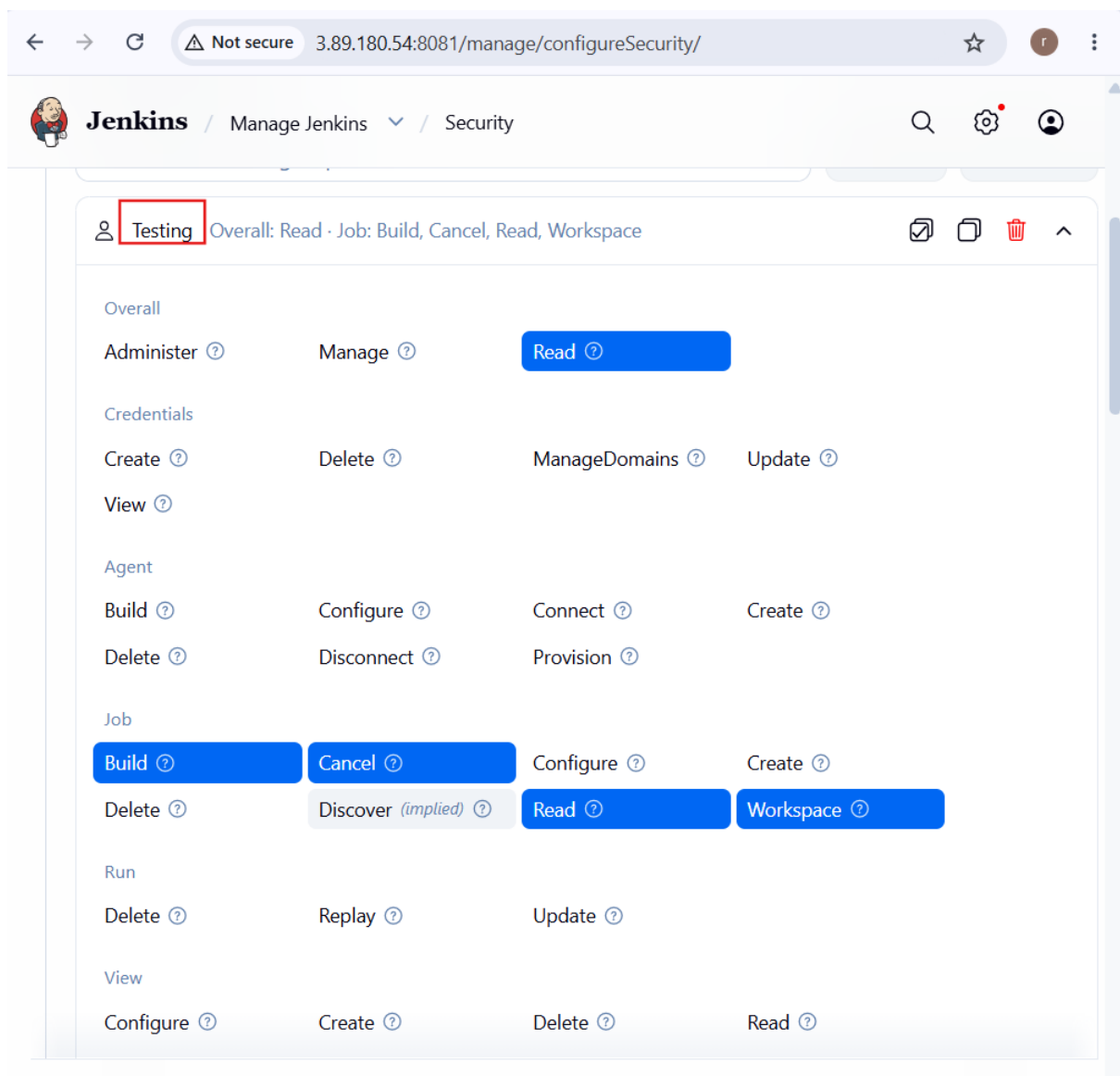
Select the Testing user.

Step 11: Give Limited Testing Permissions

Grant permissions appropriate for testing activities:

- Overall → Read ✓
- Job → Read ✓
- Job → Build ✓
- Job → Workspace ✓
- Job → Cancel ✓

Do not provide administrative permissions.



Step 12: Verify Administrator Access

Make sure your current administrator account still has:

Overall → Administer 

This is important because removing administrator access accidentally can lock you out of Jenkins.

Step 13: Save the Security Configuration

Scroll to the bottom of the page and click:

Save

Jenkins will now apply the new authorization settings.

Step 14: Test the DevOps User

Log out from the administrator account and log in as:

DevOps

Verify that the user can access and build the required Jenkins jobs but **cannot access administrative configuration**.



⚠ Not secure 3.89.180.54:8081/login?from=%2F



Sign in to Jenkins

Username

Devops

Password

.....

☐

Keep me signed in

Sign in



Devops



Profile



Builds



My Views



Account



Appearance



Preferences



Security



Experiments



Credentials

 Add description

Username: Devops

Step 15: Test the Testing User

Log out and log in as: Testing

Sign in to Jenkins

Username

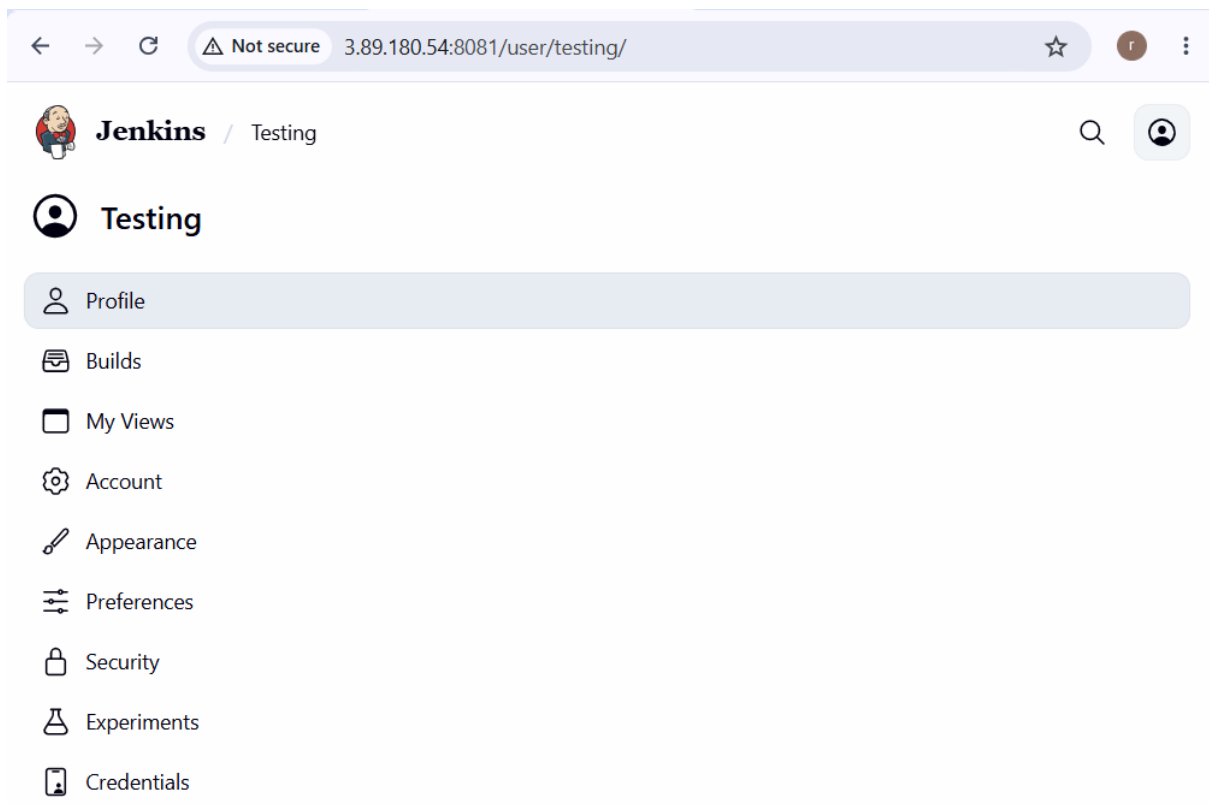
Testing

Password

.....

☐ Keep me signed in

Sign in



Verify that the user can view and execute the required jobs but cannot perform administrative operations such as changing Jenkins security settings, managing users, or configuring credentials.

Expected Result

User	Access Level	Main Permissions
Administrator	Full Access	Manage Jenkins, users, security, credentials, jobs
DevOps	Limited Access	Read, Build, Workspace, Cancel
Testing	Limited Access	Read, Build, Workspace, Cancel

4. Configure labels and restrict the jobs to execute based on label only.

The goal is to create labels such as devops-node and testing-node and ensure that a job runs **only on the node matching its configured label**.

Step 1: Open Jenkins

Log in to Jenkins using an account with **Administrator** privileges.

Step 2: Open Manage Jenkins

From the Jenkins Dashboard, click:

Manage Jenkins

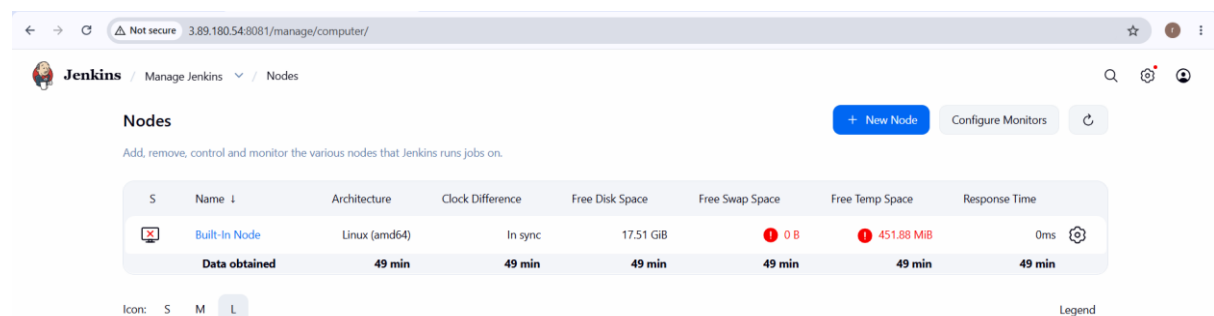
Step 3: Open Nodes

Click:

Nodes

Depending on your Jenkins version, this may appear as:

Manage Jenkins → Nodes

A screenshot of the Jenkins web interface showing the 'Nodes' page. The browser address bar shows '3.89.180.54:8081/manage/computer/'. The page title is 'Nodes'. Below the title, there's a description: 'Add, remove, control and monitor the various nodes that Jenkins runs jobs on.' There are three buttons: '+ New Node', 'Configure Monitors', and a refresh icon. A table lists the nodes. The first node is 'Built-In Node' with architecture 'Linux (amd64)', clock difference 'In sync', free disk space '17.51 GiB', free swap space '0 B', free temp space '451.88 MiB', and response time '0ms'. Below the table, there's a legend with icons for 'S', 'M', and 'L'.

Step 4: Select the Jenkins Node

Select the node on which you want to run the job.

For example:

Built-In Node

or an existing agent such as:

DevOps-Agent

Step 5: Open Node Configuration

Click:

Configure

This opens the configuration page for the selected node.

Step 6: Configure the Node Label

Locate the:

Labels

field.

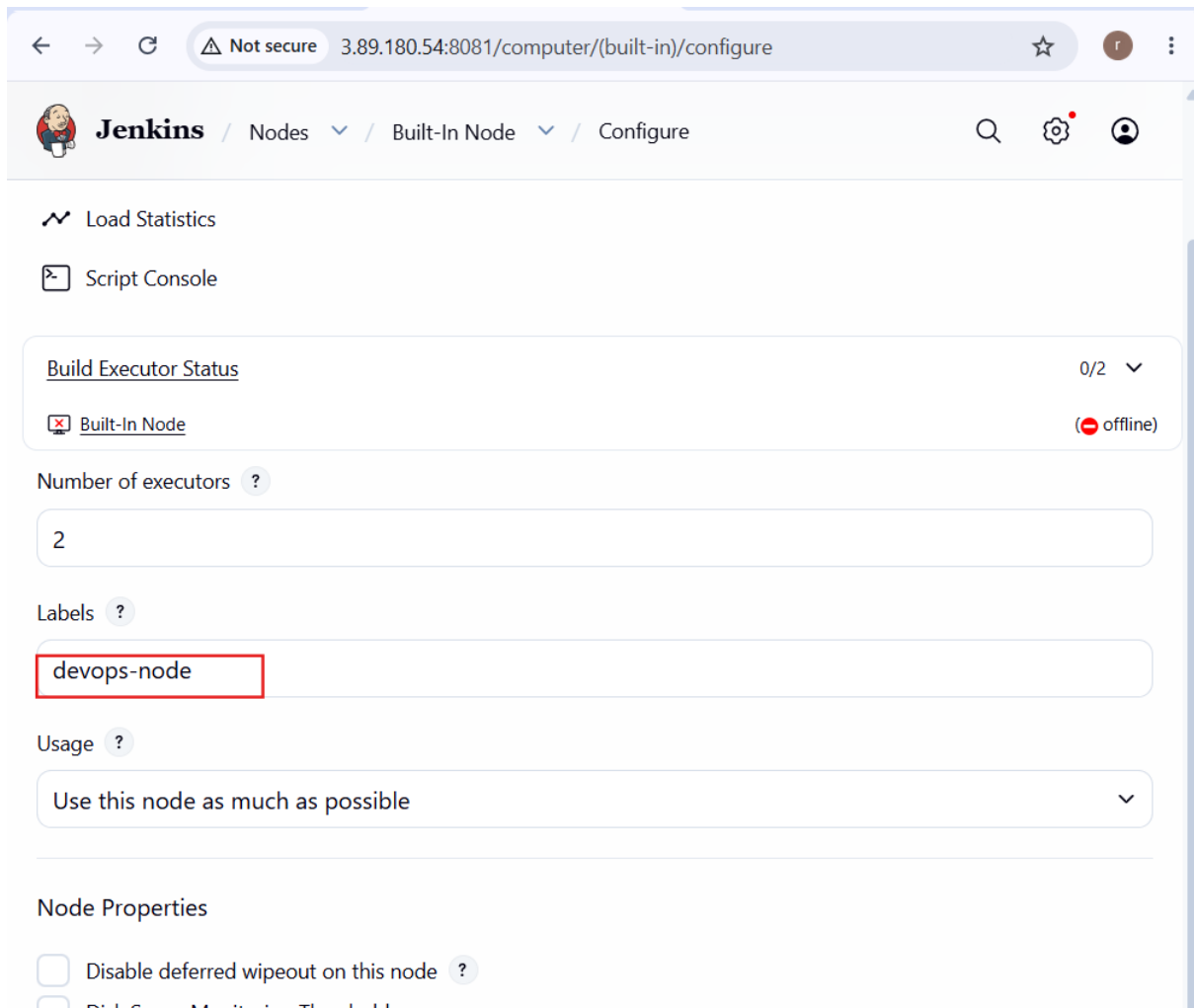
Enter a label such as:

devops-node

If you have a separate testing agent, configure its label as:

testing-node

Labels are used by Jenkins to identify which node should execute a particular job.



Step 8: Create or Open a Jenkins Job

Go back to the Jenkins Dashboard.

Create a new **Freestyle project** or open an existing job.

For example:

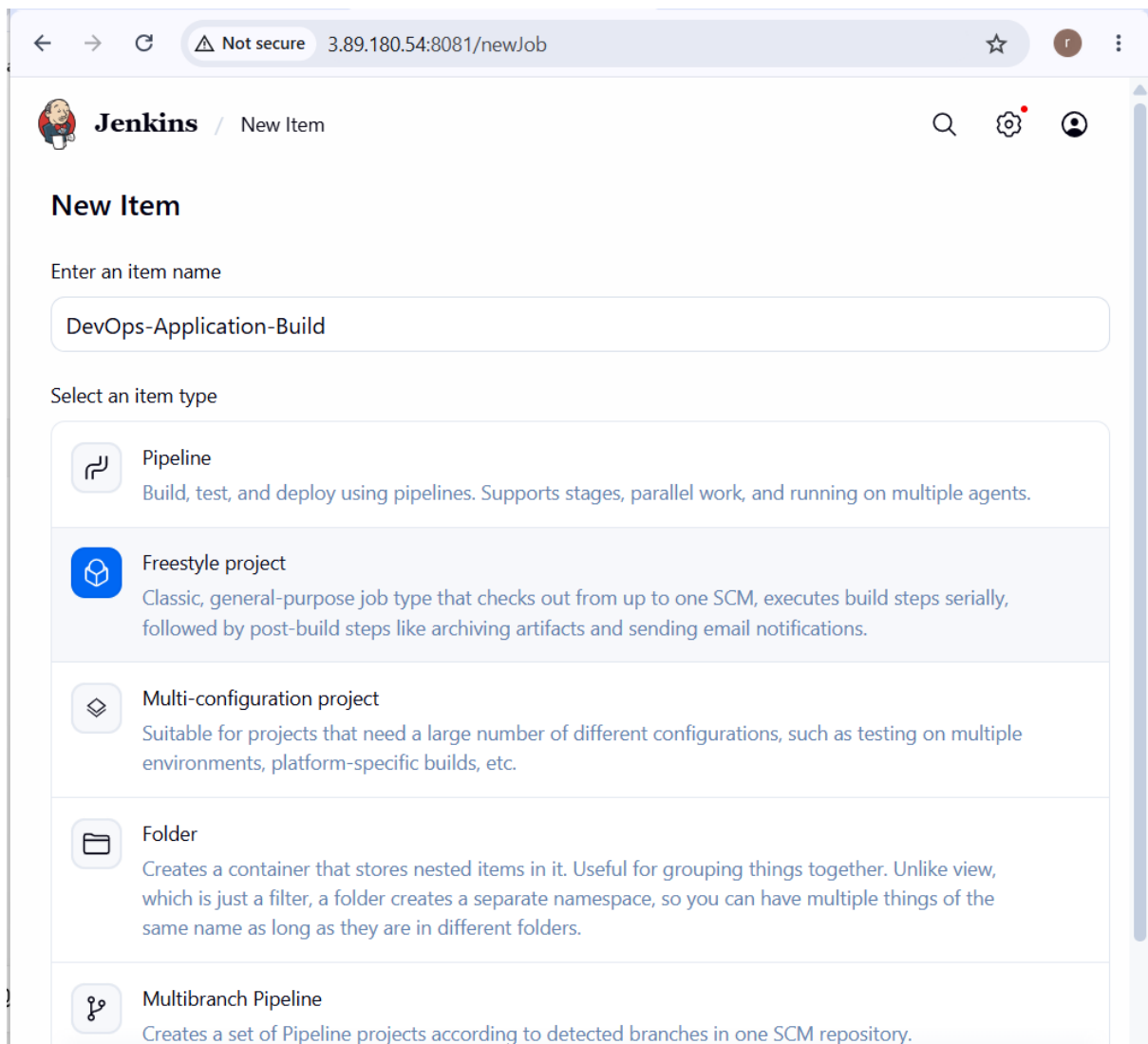
DevOps-Application-Build

Step 9: Open Job Configuration

Inside the job, click:

Configure

Scroll to the **General** section.



Step 10: Enable Label Restriction

Find and select:

Restrict where this project can be run

A text box called **Label Expression** will appear.

Step 11: Enter the Required Label

Enter:

devops-node

This tells Jenkins that the job must execute on a node having the devops-node label.

Step 12: Save the Job

Click:

Save

The job is now restricted to the specified label.

The screenshot shows the Jenkins web interface in a browser. The address bar indicates the URL is `3.89.180.54:8081/job/DevOps-Application-Build/configure`. The page title is "Jenkins / DevOps-Application-Build / Configure". The "General" tab is selected, and the job is "Enabled". A large text area for the "Description" is empty. Below it, there are several checkboxes: "Discard old builds", "GitHub project", "This project is parameterized", "Throttle builds", and "Execute concurrent builds if necessary", all of which are unchecked. The "Restrict where this project can be run" checkbox is checked and highlighted with a red rectangle. Below this checkbox, the "Label Expression" is set to "devops-node". A message at the bottom states: "Label devops-node matches 1 node. Permissions or other restrictions provided by plugins may further reduce that list."

← → ↻ ⚠ Not secure 3.89.180.54:8081/job/DevOps-Application-Build/configure ☆ ⓘ

Jenkins / DevOps-Application-Build ▾ / Configure 🔍 ⚙️ 👤

General Enabled

Description

Plain text [Preview](#)

☐ Discard old builds ⓘ

☐ GitHub project

☐ This project is parameterized ⓘ

☐ Throttle builds ⓘ

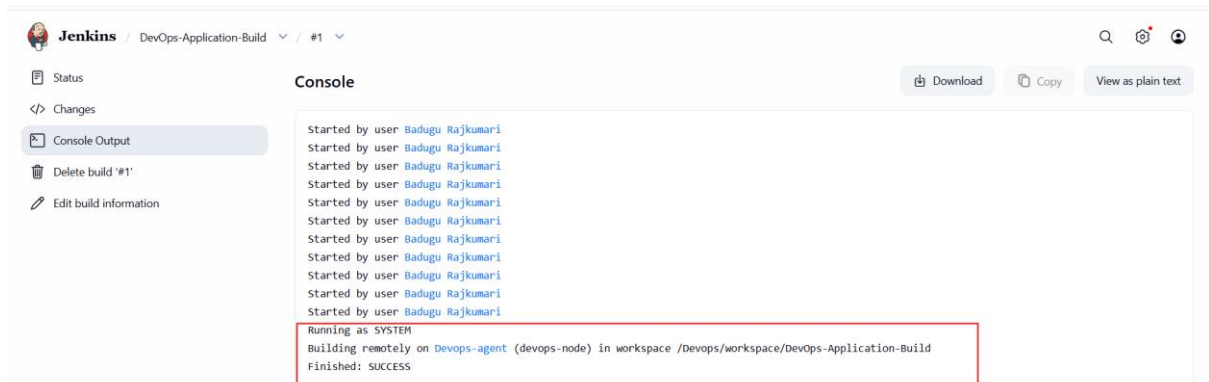
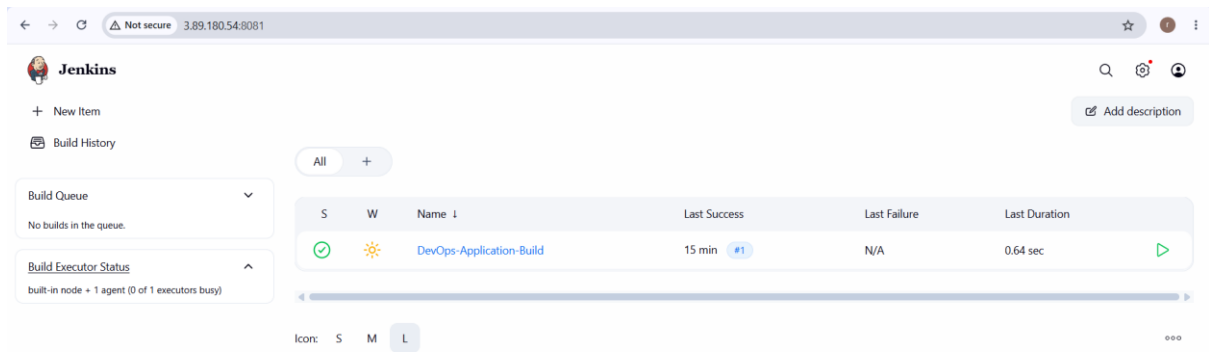
☐ Execute concurrent builds if necessary ⓘ

☒ Restrict where this project can be run ⓘ

Label Expression ⓘ

devops-node

Label devops-node matches 1 node. Permissions or other restrictions provided by plugins may further reduce that list.



5. Create Three sample jobs using the below URL.

- **Source Code:** https://github.com/betawins/Techie_horizon_Login_app.git

NOTE: First, Need to Install git to our EC2

Job	Purpose
DevOps-App-Build	Clone and build/check the application
DevOps-App-Test	Clone and test the application
DevOps-App-Deploy	Clone and prepare/deploy the application

Job 1 — DevOps-App-Build

Step 1

Go to: **Jenkins Dashboard** → **New Item**

Step 2

Enter: DevOps-Application-Build

Select: **Freestyle project**

Click **OK**.

Step 3 — Restrict the job to your agent

Under **General**, select:

Restrict where this project can be run

Enter:

devops-node

This ensures the job runs on your Devops-agent.

The screenshot shows the Jenkins configuration interface for a job named 'DevOps-App-Build'. The browser address bar indicates the URL is '3.89.180.54:8081/job/DevOps-App-Build/configure'. The left sidebar shows the 'Configure' menu with options: General (selected), Source Code Management, Triggers, Environment, Build Steps, and Post-build Actions. The main content area is titled 'Configure' and has a 'Restrict where this project can be run' checkbox checked. Below this, the 'Label Expression' is set to 'devops-node', with a note stating 'Label devops-node matches 2 nodes. Permissions or other restrictions provided by plugins may further reduce that list.' An 'Advanced' dropdown is visible. The 'Source Code Management' section is expanded, showing 'Git' selected as the provider. The 'Repository URL' is set to 'https://github.com/betawins/Techie_horizon_Login_app.git'. At the bottom, there are 'Save' and 'Apply' buttons.

←

→

↻

Not secure 3.89.180.54:8081/job/Devops-App-Build/configure

☆

🔔

 Jenkins

Devops-App-Build ▾ / Configure

🔍

⚙️

👤

Configure

⚙️ General

📁 Source Code Management

🕒 Triggers

🌐 Environment

📋 Build Steps

🔧 Post-build Actions

Build Steps

Automate your build process with ordered tasks like code compilation, testing, and deployment.

⋮ Execute shell ?

Command

See [the list of available environment variables](#)

```
echo "Workspace:"
pwd

echo "Files available for deployment:"
ls -la
```

Advanced ▾

+ Add build step

Post-build Actions

Save

Apply

←

→

↻

Not secure 3.89.180.54:8081

☆

🔔

⋮

 Jenkins

+ New Item

📋 Build History

Build Queue

No builds in the queue.

Build Executor Status

📋 Built-in Node

0/2

📋 DevOps-agent

0/1

All +

S	W	Name ↓	Last Success	Last Failure	Last Duration	
🟢	☀️	Devops-App-Build	22 sec #1	N/A	3.7 sec	▶️
🟢	☀️	DevOps-Application-Build	33 min #1	N/A	0.64 sec	▶️

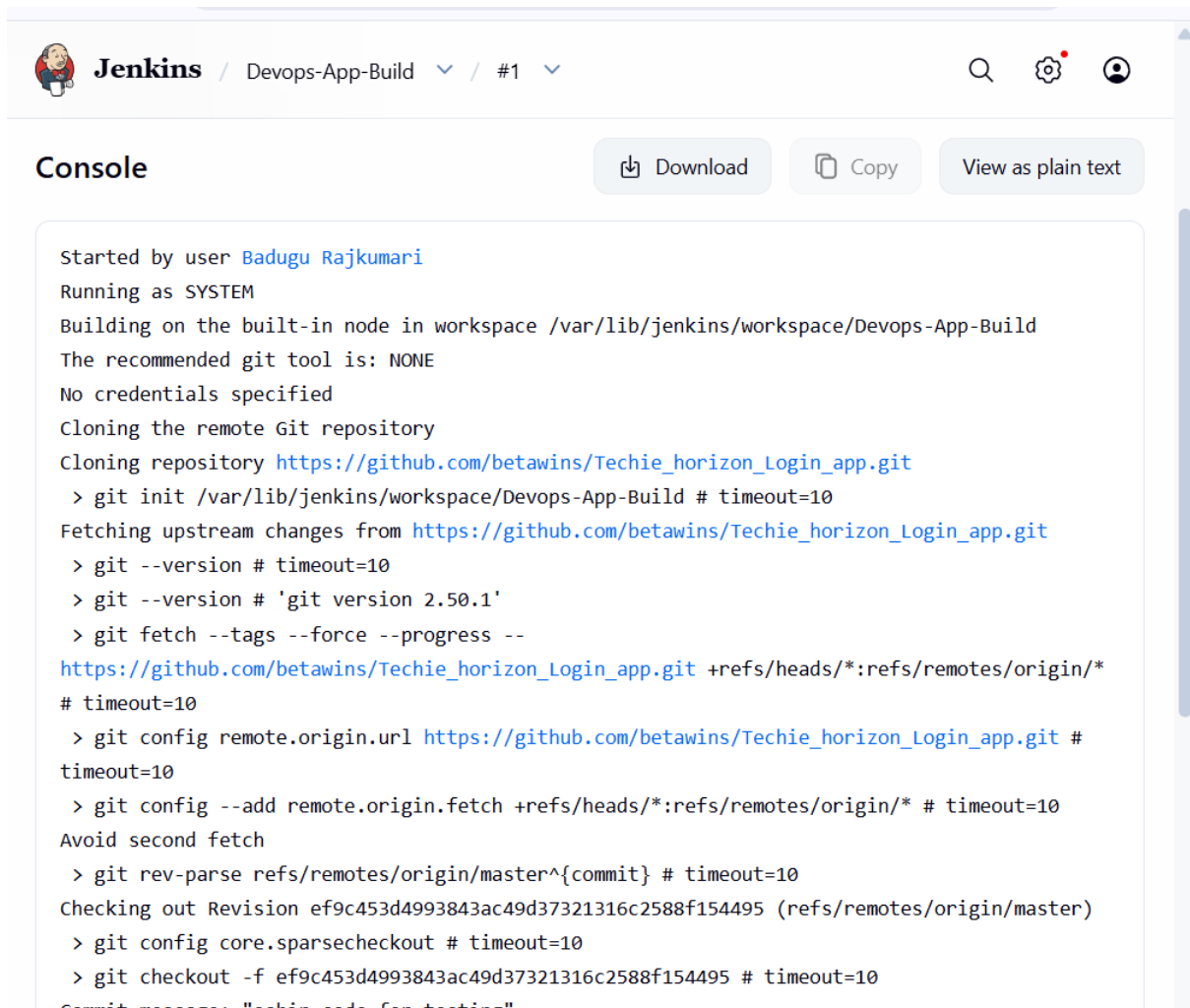
Icon: S M L

🔍

⚙️

👤

Add description



The screenshot shows the Jenkins web interface. At the top, the Jenkins logo is on the left, followed by the breadcrumb 'Devops-App-Build' and a dropdown menu showing '#1'. On the right, there are icons for search, settings, and a user profile. Below the header, the 'Console' tab is selected, with buttons for 'Download', 'Copy', and 'View as plain text'. The console output displays the execution of a Jenkins job, starting with the user 'Badugu Rajkumari' and running as 'SYSTEM'. It details the cloning of a Git repository from 'https://github.com/betawins/Techie_horizon_Login_app.git' and the execution of various Git commands to fetch and checkout the code. The final commit message is 'sahin code for testing'.

```
Started by user Badugu Rajkumari
Running as SYSTEM
Building on the built-in node in workspace /var/lib/jenkins/workspace/Devops-App-Build
The recommended git tool is: NONE
No credentials specified
Cloning the remote Git repository
Cloning repository https://github.com/betawins/Techie_horizon_Login_app.git
> git init /var/lib/jenkins/workspace/Devops-App-Build # timeout=10
Fetching upstream changes from https://github.com/betawins/Techie_horizon_Login_app.git
> git --version # timeout=10
> git --version # 'git version 2.50.1'
> git fetch --tags --force --progress --
https://github.com/betawins/Techie_horizon_Login_app.git +refs/heads/*:refs/remotes/origin/*
# timeout=10
> git config remote.origin.url https://github.com/betawins/Techie_horizon_Login_app.git #
timeout=10
> git config --add remote.origin.fetch +refs/heads/*:refs/remotes/origin/* # timeout=10
Avoid second fetch
> git rev-parse refs/remotes/origin/master^{commit} # timeout=10
Checking out Revision ef9c453d4993843ac49d37321316c2588f154495 (refs/remotes/origin/master)
> git config core.sparsecheckout # timeout=10
> git checkout -f ef9c453d4993843ac49d37321316c2588f154495 # timeout=10
Commit message: "sahin code for testing"
```

Job 2 — DevOps-Application-Test

Go to:

Jenkins Dashboard → New Item

Create:

DevOps-Application-Test

Select:

Freestyle project → OK

Step 1 — Agent

Enable:

Restrict where this project can be run

Enter: devops-node

Step 2 — Git

Select:

Source Code Management → Git

Repository:

https://github.com/betawins/Techie_horizon_Login_app.git

Branch: */master

← → ↻

⚠ Not secure

3.89.180.54:8081/job/Devops-App-Test/

 **Jenkins** / Devops-App-Test

Status

</> Changes

Workspace

▶ Build Now

🗑 Delete Project

⚙ Configure

✎ Rename

Builds > ...

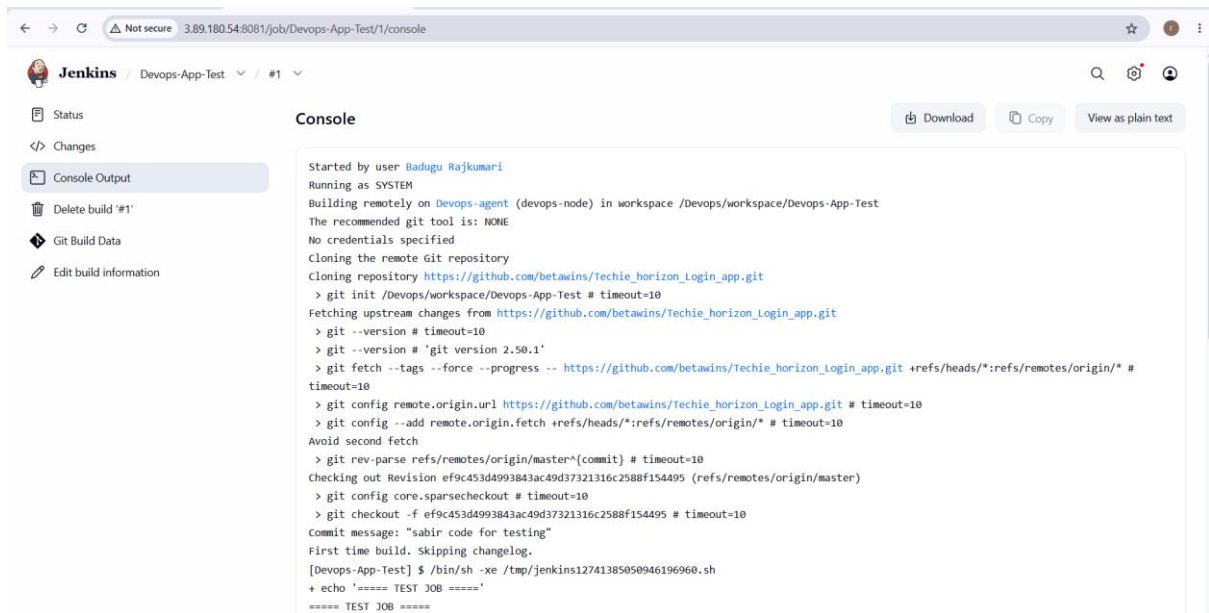
Today

✓ #1 11:06 AM

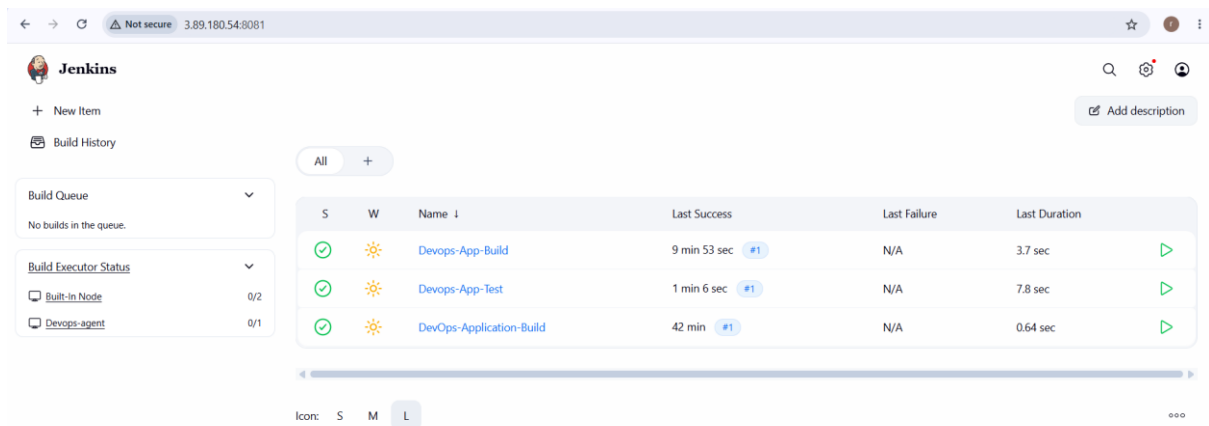
▼

Devops-App-Test

Permalinks



```
Started by user Badugu Rajkumari
Running as SYSTEM
Building remotely on Devops-agent (devops-node) in workspace /Devops/workspace/Devops-App-Test
The recommended git tool is: NONE
No credentials specified
Cloning the remote Git repository
Cloning repository https://github.com/betawins/Techie_horizon_Login_app.git
> git init /Devops/workspace/Devops-App-Test # timeout=10
Fetching upstream changes from https://github.com/betawins/Techie_horizon_Login_app.git
> git --version # timeout=10
> git --version # 'git version 2.50.1'
> git fetch --tags --force --progress -- https://github.com/betawins/Techie_horizon_Login_app.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> git config remote.origin.url https://github.com/betawins/Techie_horizon_Login_app.git # timeout=10
> git config --add remote.origin.fetch +refs/heads/*:refs/remotes/origin/* # timeout=10
Avoid second fetch
> git rev-parse refs/remotes/origin/master^{commit} # timeout=10
Checking out Revision ef9c453d4993843ac49d37321316c2588f154495 (refs/remotes/origin/master)
> git config core.sparsecheckout # timeout=10
> git checkout -f ef9c453d4993843ac49d37321316c2588f154495 # timeout=10
Commit message: "sabir code for testing"
First time build. Skipping changelog.
[Devops-App-Test] $ /bin/sh -xe /tmp/jenkins12741385050946196960.sh
+ echo '==== TEST JOB ====='
===== TEST JOB =====
```



S	W	Name	Last Success	Last Failure	Last Duration
✓	☀	Devops-App-Build	9 min 53 sec #1	N/A	3.7 sec
✓	☀	Devops-App-Test	1 min 6 sec #1	N/A	7.8 sec
✓	☀	DevOps-Application-Build	42 min #1	N/A	0.64 sec

Job 3 — DevOps-Application-Deploy

Create another:

Jenkins Dashboard → New Item

Name:

DevOps-Application-Deploy

Select:

Freestyle project

Click OK.

Step 1 — Agent

Enable:

Restrict where this project can be run

Enter:

devops-node

Step 2 — Git

Repository:

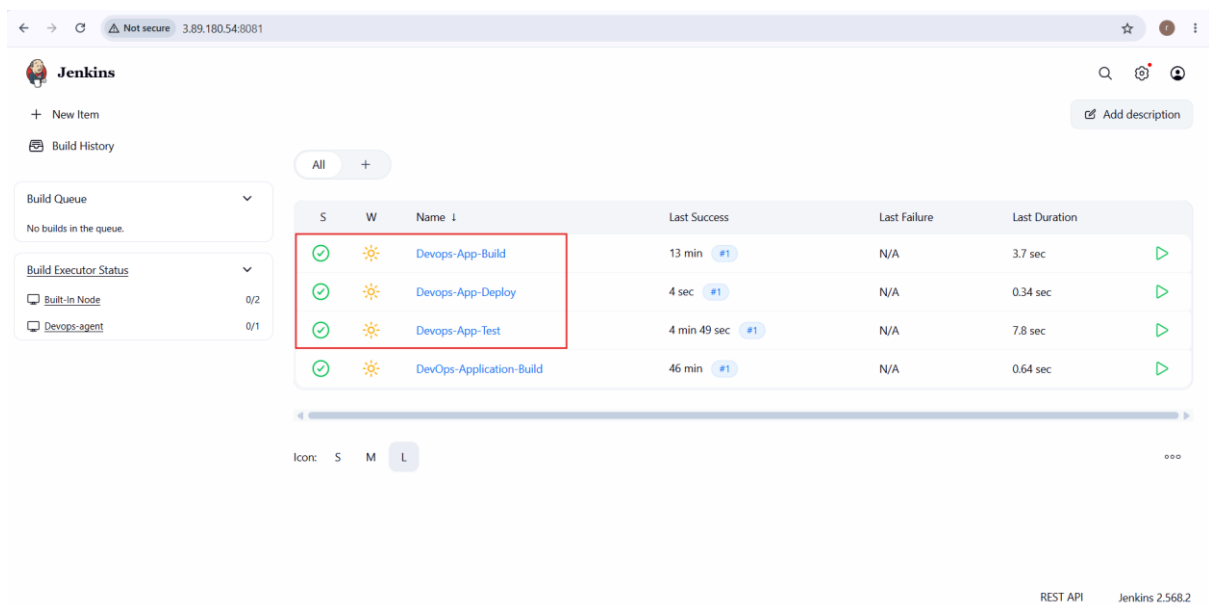
https://github.com/betawins/Techie_horizon_Login_app.git

Branch:

*/main

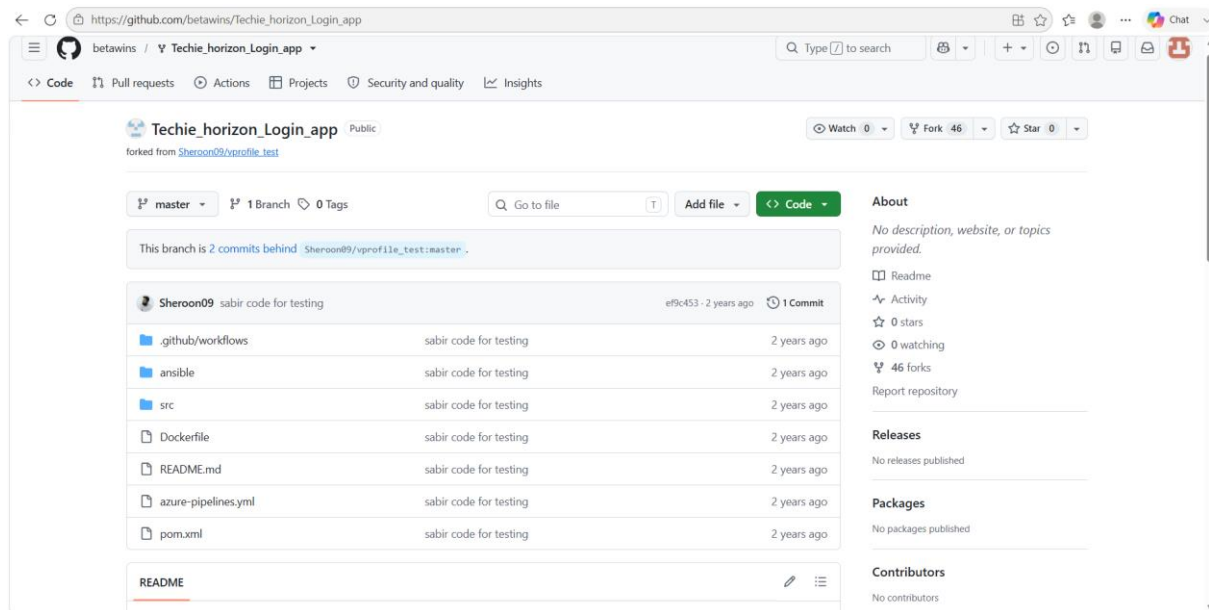
Step 3 — Execute shell

For your practice documentation, initially use:



The screenshot shows the Jenkins web interface. The top navigation bar includes the Jenkins logo, a search icon, and a user profile icon. The left sidebar contains a 'Build Queue' section with 'No builds in the queue.' and a 'Build Executor Status' section showing 'Built-in Node' with 0/2 and 'DevOps-agent' with 0/1. The main content area displays a table of builds. The first three rows are highlighted with a red box: 'DevOps-App-Build', 'DevOps-App-Deploy', and 'DevOps-App-Test'. The table has columns for status (S), workspace (W), name, last success, last failure, last duration, and a play button icon. The bottom right corner shows 'REST API' and 'Jenkins 2.568.2'.

S	W	Name	Last Success	Last Failure	Last Duration
✓	☀	DevOps-App-Build	13 min #1	N/A	3.7 sec
✓	☀	DevOps-App-Deploy	4 sec #1	N/A	0.34 sec
✓	☀	DevOps-App-Test	4 min 49 sec #1	N/A	7.8 sec
✓	☀	DevOps-Application-Build	46 min #1	N/A	0.64 sec



```

root@ip-172-31-19-206:/Devops/Techie_horizon_Login_app
[root@ip-172-31-19-206 ~]# cd /Devops
git clone https://github.com/betawins/Techie_horizon_Login_app.git
cd Techie_horizon_Login_app
ls -la
Cloning into 'Techie_horizon_Login_app'...
remote: Enumerating objects: 86, done.
remote: Total 86 (delta 0), reused 0 (delta 0), pack-reused 86 (from 1)
Receiving objects: 100% (86/86), 704.53 KiB | 28.18 MiB/s, done.
Resolving deltas: 100% (3/3), done.
total 20
drwxr-xr-x. 6 root    root    136 Aug 11 11:13 .
drwxr-xr-x. 5 ec2-user ec2-user 88 Aug 11 11:13 ..
drwxr-xr-x. 7 root    root    147 Aug 11 11:13 .git
drwxr-xr-x. 3 root    root    23 Aug 11 11:13 .github
-rw-r--r--. 1 root    root    137 Aug 11 11:13 Dockerfile
-rw-r--r--. 1 root    root    978 Aug 11 11:13 README.md
drwxr-xr-x. 2 root    root    91 Aug 11 11:13 ansible
-rw-r--r--. 1 root    root    508 Aug 11 11:13 azure-pipelines.yml
-rw-r--r--. 1 root    root    5830 Aug 11 11:13 pom.xml
drwxr-xr-x. 4 root    root    30 Aug 11 11:13 src
[root@ip-172-31-19-206 Techie_horizon_Login_app]#

```

6. Create one Jenkins job using GitHub Private repository.

1. Create a Private GitHub Repository

In GitHub:

1. Sign in to GitHub.
2. Click **New repository**.
3. Enter a repository name, for example:

Jenkins-Private-App

4. Select **Private**.
5. Click **Create repository**.

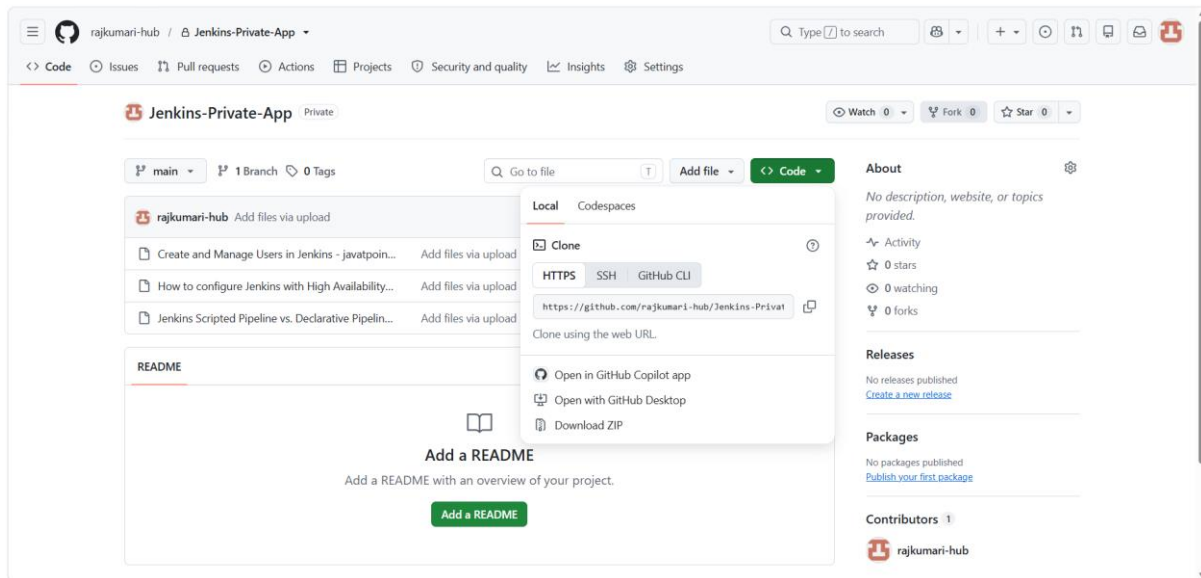
6. Upload your application files to the private repository, or push your existing application.

Your repository URL will look like:

<https://github.com/<your-username>/Jenkins-Private-App.git>

The screenshot shows the 'Create a new repository' page on GitHub. The 'General' tab is active. The 'Owner' is 'rajkumari-hub' and the 'Repository name' is 'Jenkins-Private-App'. A green checkmark indicates the name is available. The 'Description' field is empty. The 'Configuration' section shows 'Choose visibility' set to 'Private' (highlighted with a red box), 'Add README' set to 'Off', and 'Add .gitignore' set to 'No .gitignore'.

The screenshot shows the repository page for 'Jenkins-Private-App' under the 'rajkumari-hub' account. The repository is marked as 'Private'. The page includes a 'Quick setup' section with instructions for setting up the repository on a desktop or using HTTPS/SSSH. It also provides a command-line setup for creating a new repository. The repository URL is displayed as 'git@github.com:rajkumari-hub/Jenkins-Private-App.git'.



2. Create GitHub Personal Access Token

For a private repository, Jenkins needs authentication.

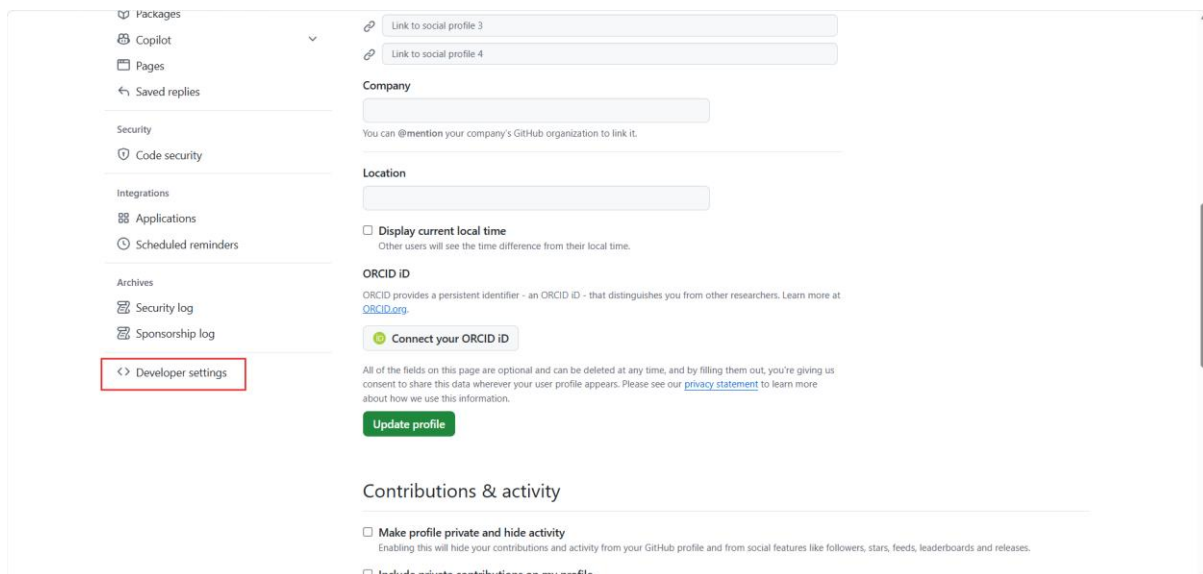
In GitHub:

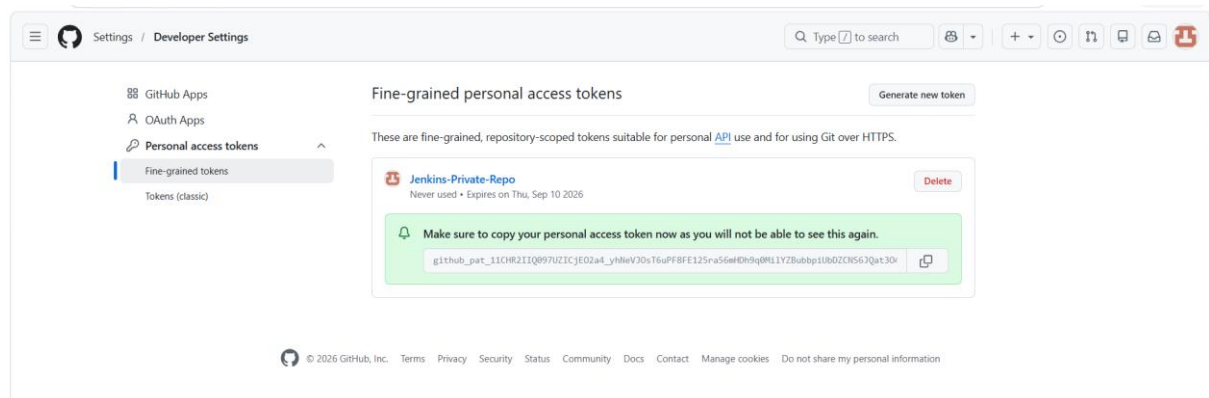
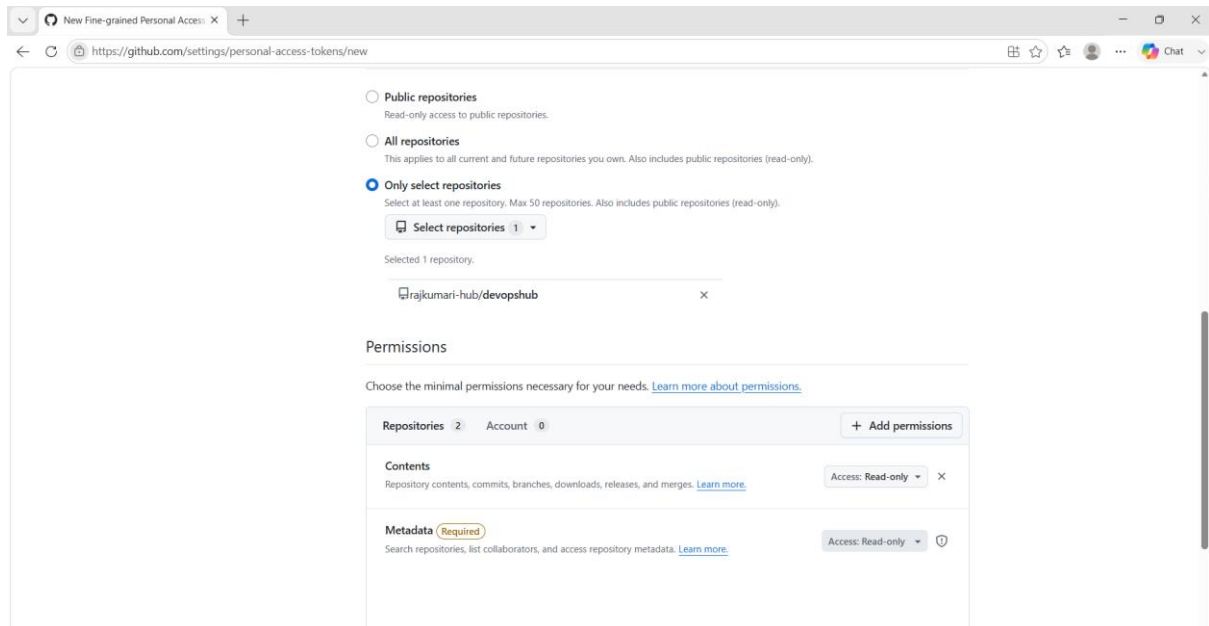
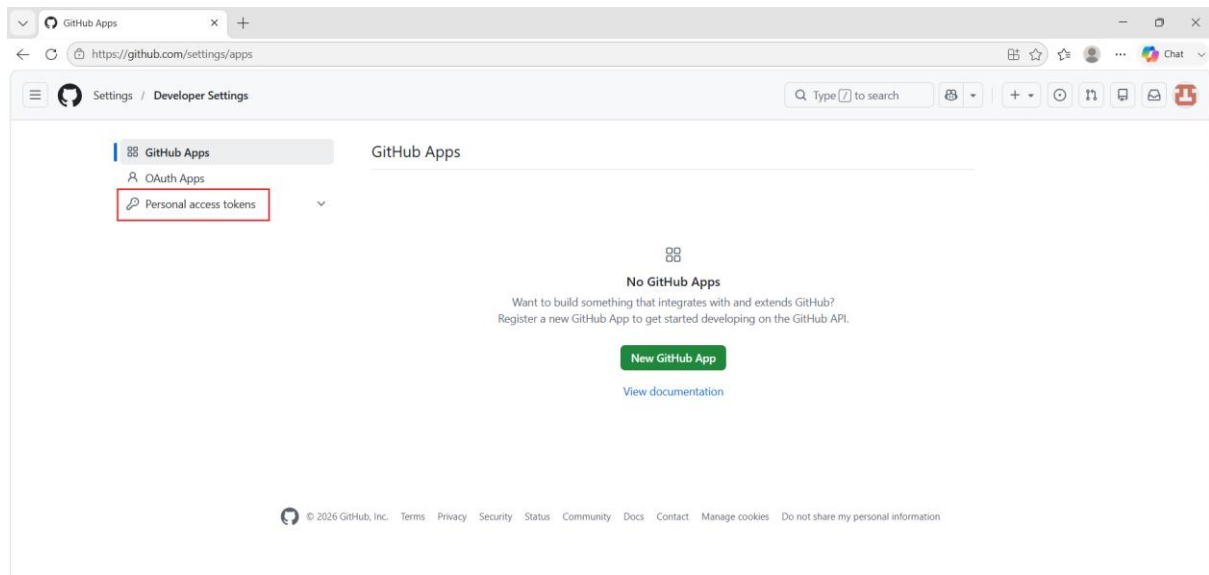
Profile → Settings → Developer settings → Personal access tokens

Create a token with permission to access your private repository.

For a fine-grained token, give it access to the required private repository and enable appropriate repository permissions, especially **Contents: Read**.

Do not share the token with anyone.





3. Add GitHub Credentials to Jenkins

Go to:

Jenkins → Manage Jenkins → Credentials

Then:

System → Global credentials → Add Credentials

Select:

Kind: Username with password

Enter:

Username: <your GitHub username>

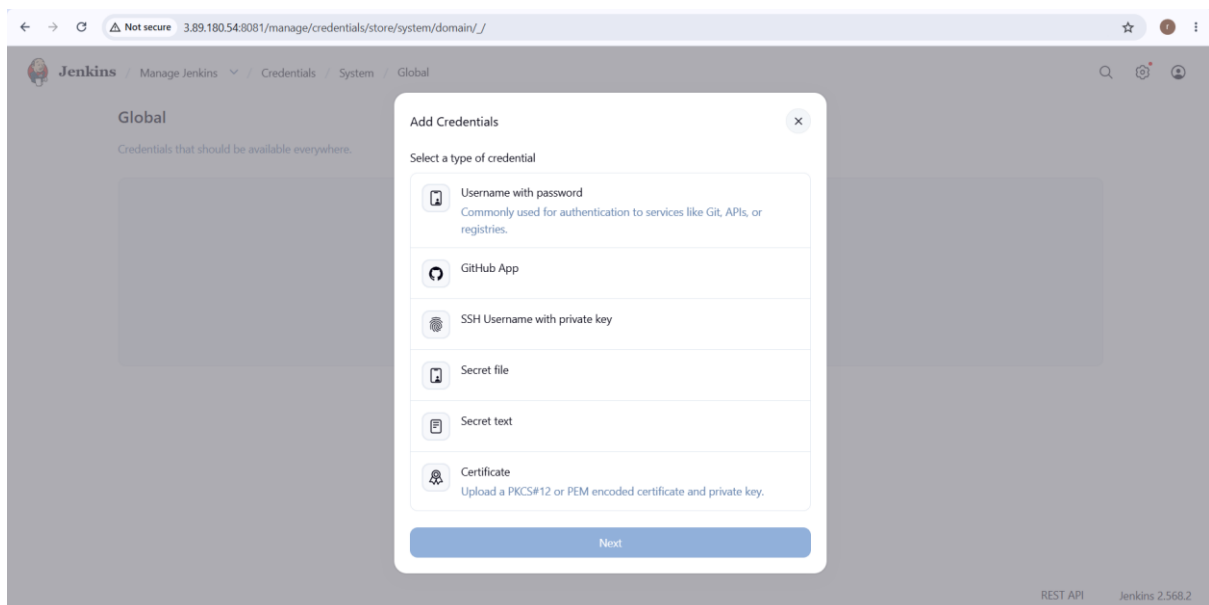
Password: <your GitHub Personal Access Token>

ID: github-private-repo

Description: GitHub Private Repository Credentials

Click:

Create



← → ↻ ⚠ Not secure 3.89.180.54:8081/manage/credentials/store/system/domain/_/ 🔑 ☆ r ⋮

 **Jenkins** / Manage Jenkins ▾ / Credentials / System / Global 🔍 ⚙️ 👤

Global

Credentials that s

⏪ Add Username with password ⏩

Scope ?
Global (Jenkins, nodes, items, all child items, etc) ▾

Username
rajkumari-hub

☐ Treat username as secret ?

Password
.....

ID ?
github-private-repo

Description ?
GitHub Private Repository Credentials

Create

 **Jenkins** / Manage Jenkins ▾ / Credentials / System / Global 🔍 ⚙️ 👤

Global

Credentials that should be available everywhere.

 **github-private-repo** rajkumari-hub/*****
GitHub Private Repository Credentials

 ⋮

4. Create the Jenkins Job

Go to:

Jenkins Dashboard → New Item

Enter:

DevOps-Private-GitHub-Build

Select:

Freestyle project

Click **OK**.

5. Configure the Agent

Under **General**, select:

Restrict where this project can be run

Enter:

devops-node

This makes the job run on your Devops-agent.

6. Configure GitHub Private Repository

Go to:

Source Code Management → Git

Enter your private repository URL:

<https://github.com/<your-username>/Jenkins-Private-App.git>

Under **Credentials**, select:

github-private-repo

For **Branch Specifier**, enter:

*/main

Jenkins will now authenticate with GitHub before cloning the private repository.

7. Add a Build Step

Go to:

Build Steps → Add build step → Execute shell

← → ↻ ⚠ Not secure 3.89.180.54:8081/job/DevOps-Private-GitHub-Build/


Jenkins / DevOps-Private-GitHub-B...

Status

</> Changes

Workspace

▶ Build Now

🗑 Delete Project

⚙ Configure

✎ Rename


DevOps-Private-GitHub-Build

Permalinks

- [Last build \(#1\), 1 min 48 sec ago](#)
- [Last stable build \(#1\), 1 min 48 sec ago](#)
- [Last successful build \(#1\), 1 min 48 sec ago](#)
- [Last completed build \(#1\), 1 min 48 sec ago](#)

Builds >

Today


#1 12:20 PM

← → ↻ ⚠ Not secure 3.89.180.54:8081


Jenkins

+ New Item

📜 Build History

🔗 Project Relationship

🔍 Check File Fingerprint

Build Queue

No builds in the queue.

Build Executor Status

🖨 Built-in Node

0/2

🖨 DevOps-agent

0/1

All +

S	W	Name ↓	Last Success	Last Failure	Last Duration
		Devops-App-Build	1 hr 24 min #1	N/A	3.7 sec 
		Devops-App-Deploy	1 hr 10 min #1	N/A	0.34 sec 
		Devops-App-Test	1 hr 15 min #1	N/A	7.8 sec 
		DevOps-Application-Build	1 hr 57 min #1	N/A	0.64 sec 
		DevOps-Private-GitHub-Build	2 min 18 sec #1	N/A	2.5 sec 

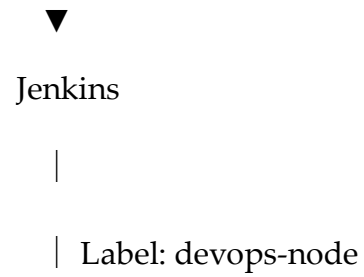
Icons: S M L

Final flow

GitHub Private Repository

|

| HTTPS + PAT



Documentation result: Jenkins successfully authenticates with a private GitHub repository using stored Jenkins credentials, checks out the source code onto the Devops-agent, and executes the configured build step.



Jenkins

/ DevOps-Private-GitHub-B...



#1



Status

</> Changes



Console Output



Delete build '#1'



Git Build Data



Edit build information

Console



Download



Copy

View as plain text

```
Started by user Badugu Rajkumari
Running as SYSTEM
Building on the built-in node in workspace /var/lib/jenkins/workspace/DevOps-Private-GitHub-Build
The recommended git tool is: NONE
using credential github-private-repo
Cloning the remote Git repository
Cloning repository https://github.com/rajkumari-hub/devopshub.git
> git init /var/lib/jenkins/workspace/DevOps-Private-GitHub-Build # timeout=10
Fetching upstream changes from https://github.com/rajkumari-hub/devopshub.git
> git --version # timeout=10
> git --version # 'git version 2.50.1'
using GIT_ASKPASS to set credentials GitHub Private Repository Credentials
> git fetch --tags --force --progress -- https://github.com/rajkumari-hub/devopshub.git
```